



Presentació Assignatura

Algorísmica Avançada | Enginyeria Informàtica

Santi Seguí | 2026-2027



Professor Agregat al departament de Matemàtiques i Informàtica
de la Universitat de Barcelona.

Doctorat amb *Visió per Computador* i *Machine Learning*
Graduat amb Informàtica

Santi Seguí **Email:** santi.segui@ub.edu



Sergio Escalera

sescalera@ub.edu



Manuel Hernández

manuel.hernandez@ub.edu



Pablo Martinez

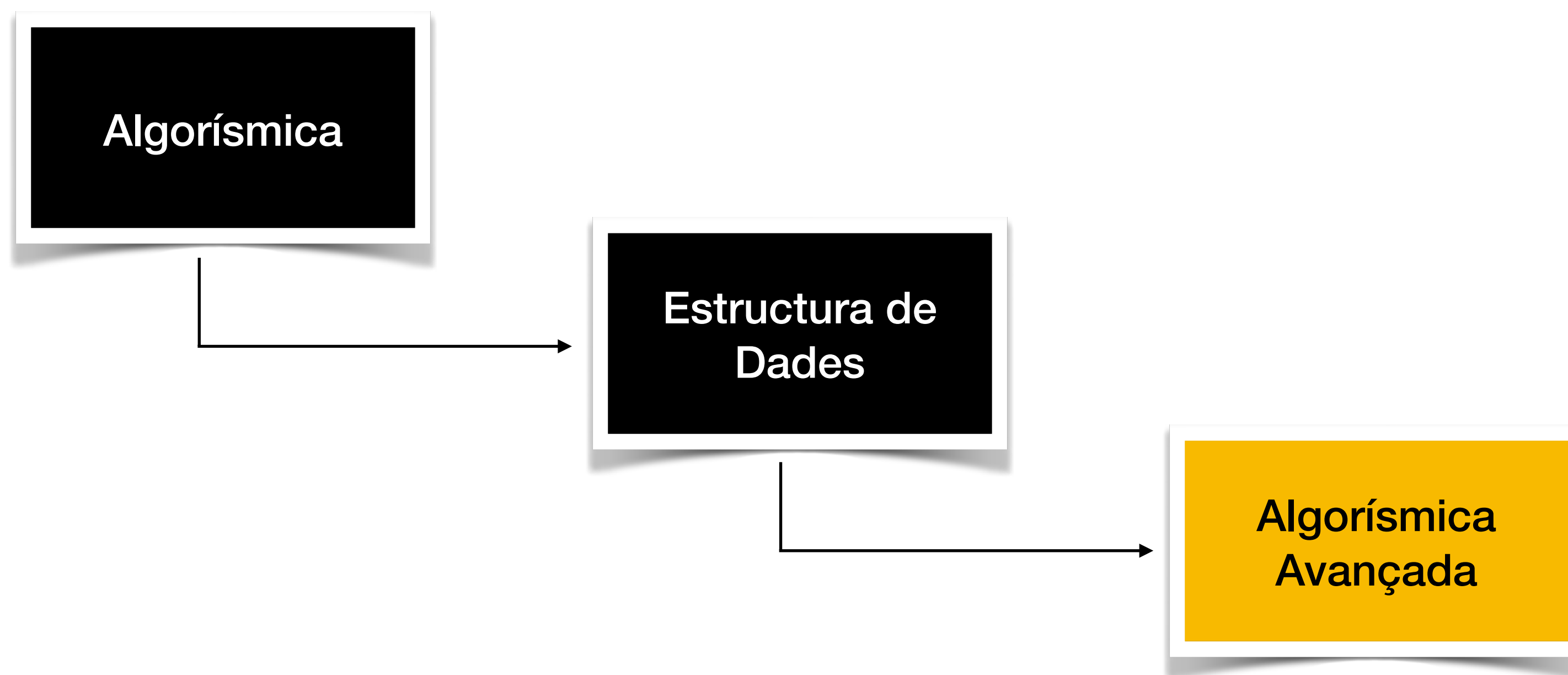
pablo.martinez@ub.edu



Oriol Pujol

oriol_pujol@ub.edu

Algorísmica Avançada



wooclap

Quina és la complexitat del següent algoritme:

```
def main(c):  
    a=1  
    b=4  
    d=max(c)  
    for i in c:  
        a=a+b  
    return a
```

$O(2c + 4)$

$O(2c)$

$O(c)$

$O(2c + 3)$

* considerem C com a la longitud de l'array

Complexitat d'un algoritme

- Abans d'estudiar algorismes concrets necessitem una manera de **mesurar com de bo és un algorisme** sense dependre de l'ordinador on s'executa. Aquesta és la idea de la **complexitat**: com creix el temps (o la memòria) a mesura que creix la mida de l'entrada.
- **Per què no cronometrar i prou?**
 - Mesurar segons en una màquina concreta no serveix per comparar algorismes: depèn del processador, del llenguatge, de la càrrega del sistema... En canvi, **comptar quantes operacions bàsiques fa un algorisme en funció de la mida de l'entrada N és independent de la màquina** i ens diu com es comportarà quan les dades creixin.
- **La pregunta clau**
 - No ens importa tant si un algorisme triga 2 o 4 segons amb dades petites, sinó **què passa quan N es fa gran**: ¿el temps es duplica, es multiplica per quatre, o explota? Això és el que captura la complexitat **asimptòtica**.

Notació asimptòtica: O , Ω i Θ

- Descriu el creixement d'una funció $T(N)$ (operacions en funció de N) ignorant constants i termes de menor ordre:
- **O (cota superior):** $T(N) = O(f(N))$ vol dir que T creix **com a molt** tan ràpid com f . És la garantia del *pitjor cas* i la notació més usada.
- **Ω (cota inferior):** $T(N) = \Omega(f(N))$ vol dir que creix com a mínim com f .
- **Θ (cota ajustada):** $T(N) = \Theta(f(N))$ vol dir totes dues coses alhora: creix **exactament** com f (llevat de constants).
- Per això escrivim $O(2n + 3) = O(n)$: descartem les constants (el «2» i el «+3») perquè no canvien la forma del creixement.

Notació Asimptòtica

Quin és el cost d'aquest algoritme?

```
def main(c):  
    a=1  
    b=4  
    d=max(c)  
    for i in c:  
        a=a+b  
    return a
```

Notació Asimptòtica

Quin és el cost d'aquest algoritme?

```
def main(c):  
    a=1           1  
    b=4           1  
    d=max(c)      c  
    for i in c:   c*(1)  
        a=a+b     1  
    return a      1  
-----  
Total = 1+1+c+c+1=2c+3  
O(c)
```

* considerem C com a la longitud de l'array

Quines respostes són correctes?

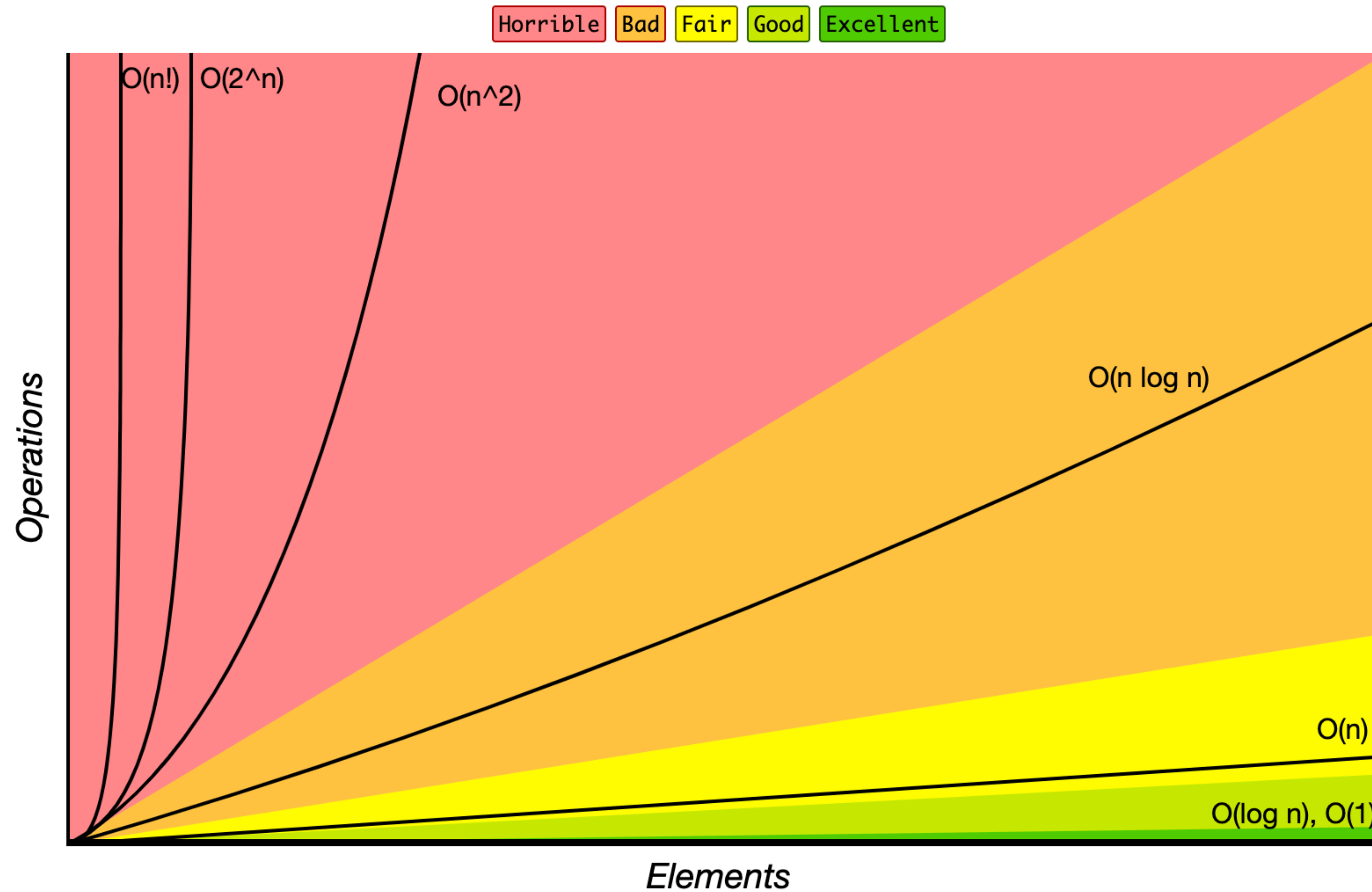
$$O(n) < O(n \log n) < O(n^2) < O(n!)$$

No podem saber si un algoritme amb complexitat $O(n!)$ és millor que un amb complexitat $O(2^n)$ sense saber el valor de n .

$$O(n) < O(2n)$$

$$O(2^n) < O(n^2)$$

Big-O Complexity Chart



Exemple: Problema de desxifrar contrasenyes

- Suposem que una contrasenya està formada per 8 lletres minúscules (a-z). Un atacant intenta endevinar-la amb força bruta.

- Quantes possibles combinacions hi ha?

- $26^8 \approx 2 * 10^{11}$

- Si l'atacant pot provar un milió de contrasenyes per segon, quant trigarà?

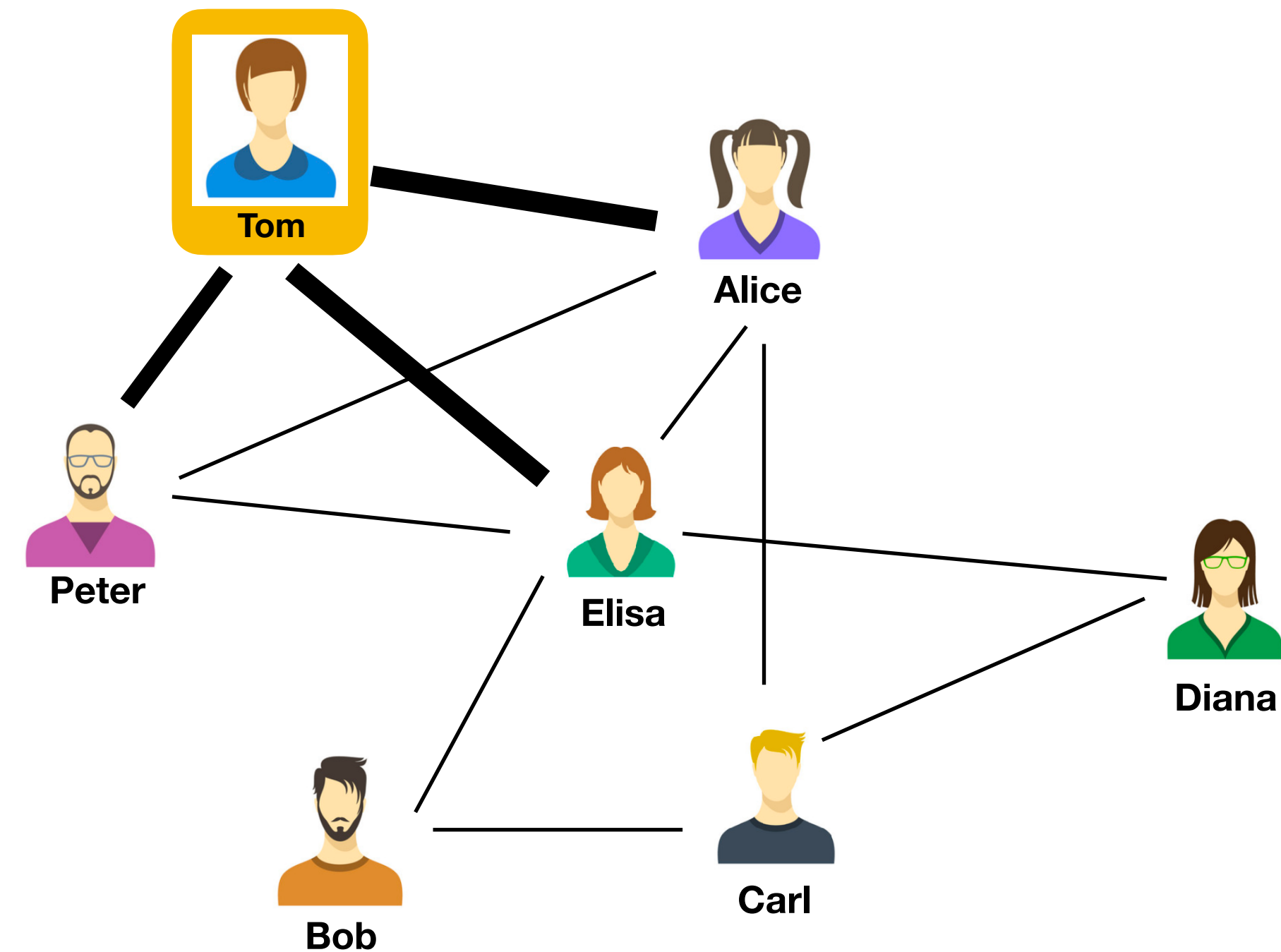
- $\frac{20^8}{10^6 \text{ proves/s}} \approx 208.000 \text{ segons} \approx 58 \text{ hores}$

- 🧠 Per què importa la complexitat?

- Per a una contrasenya de 4 caràcters es pot trencar en segons
- Per a una contrasenya de 12 caràcters necessitem més de 3.000 anys.
- Això mostra un creixement exponencial.

Podem suggerir “amics” al TOM?.

Recomanarem aquells que estiguin distància 2 (amics d'amics). Quin algoritme podem utilitzar?



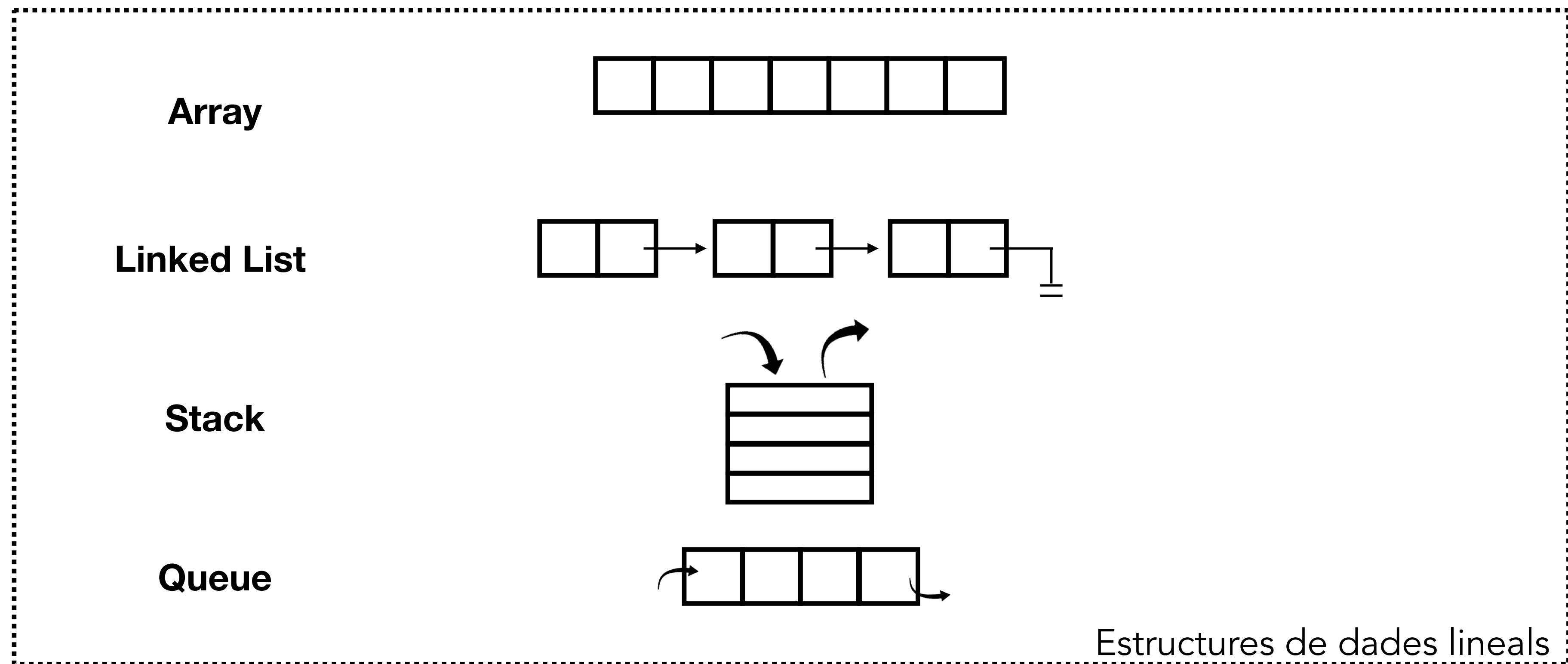
BFS

DFS

Què és un graf?

Què és un graf?

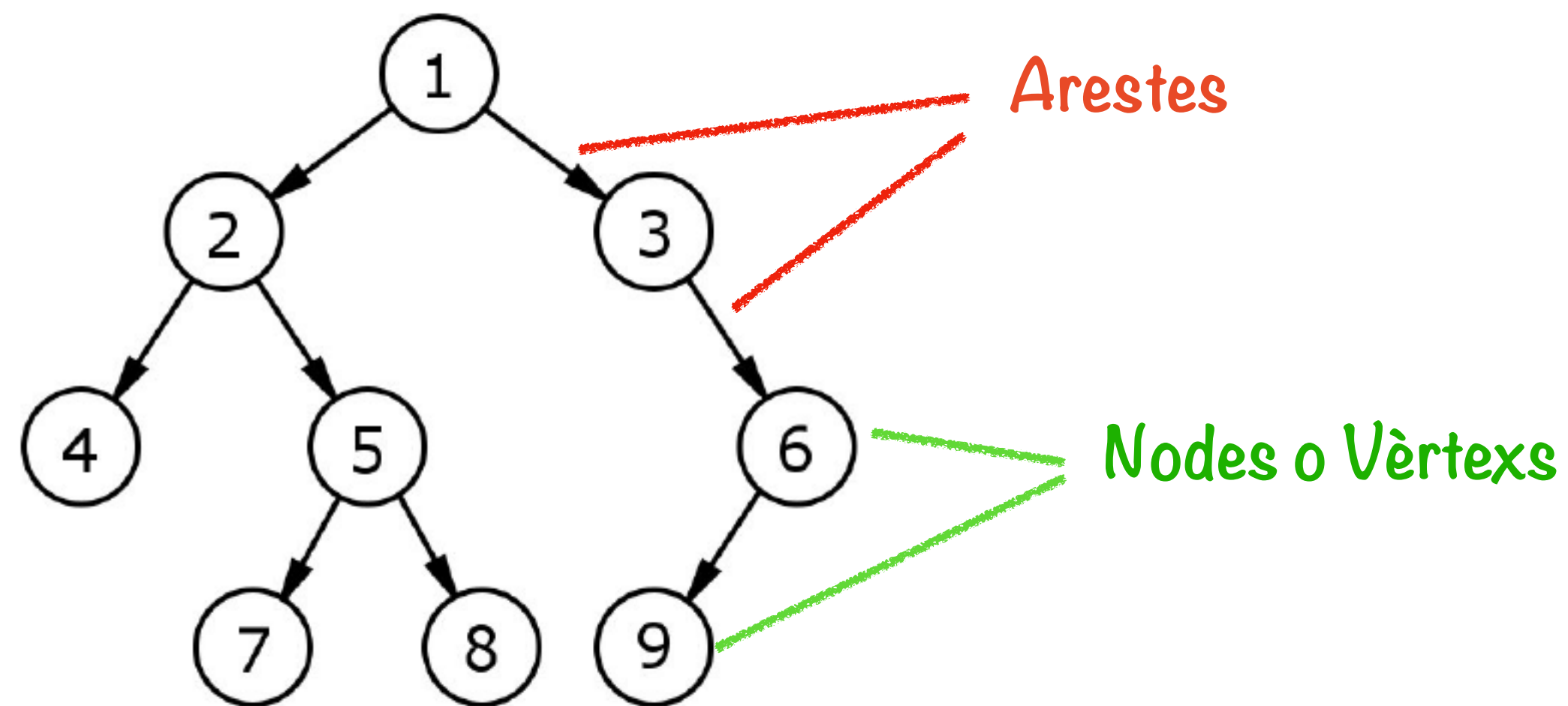
- El graf és una estructura de dades **NO LINEAL**



Què és un graf?

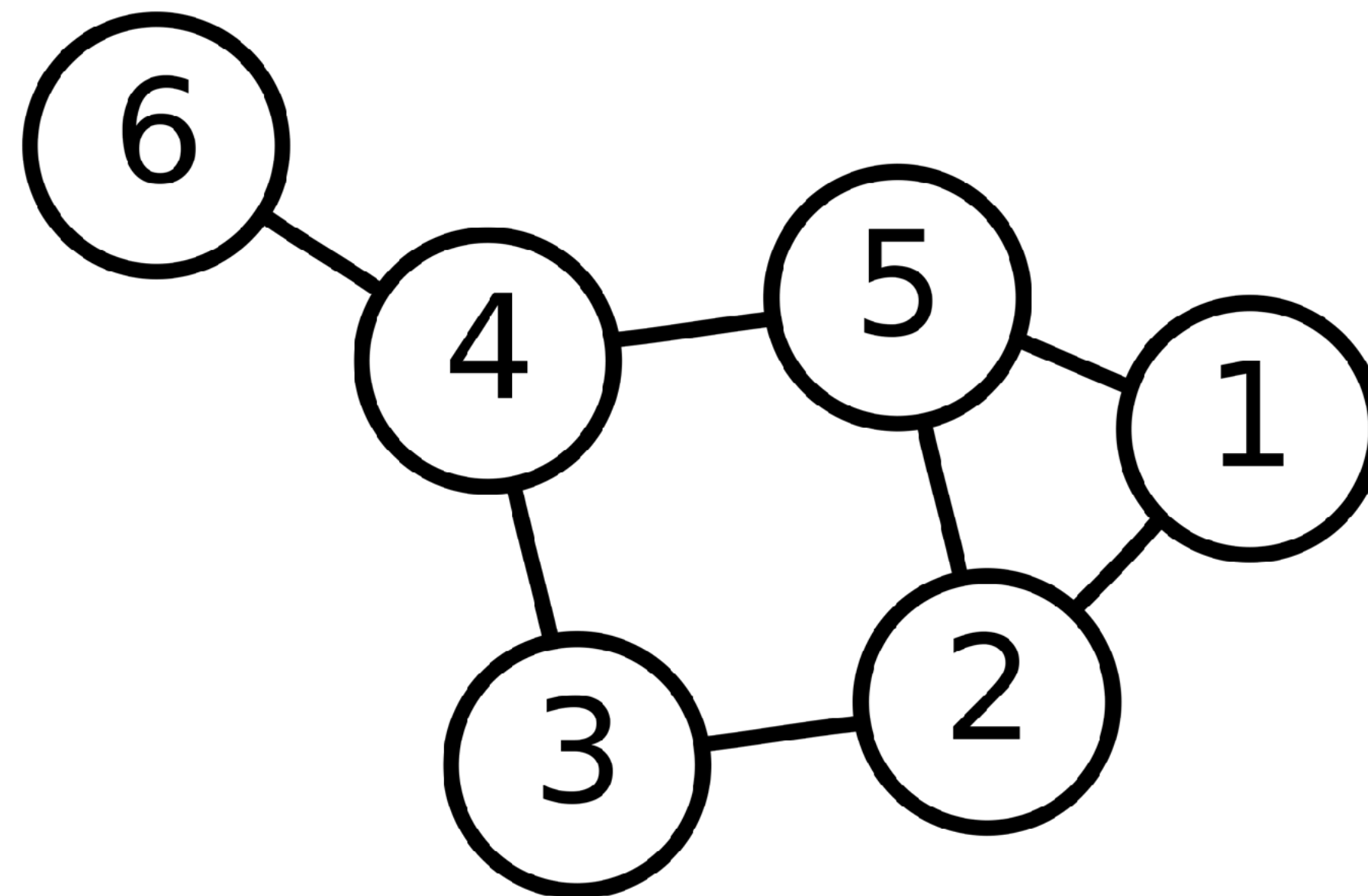
- El graf és una estructura de dades **NO LINEAL**

Quina estructura de dades **NO LINEAL** coneixeu?



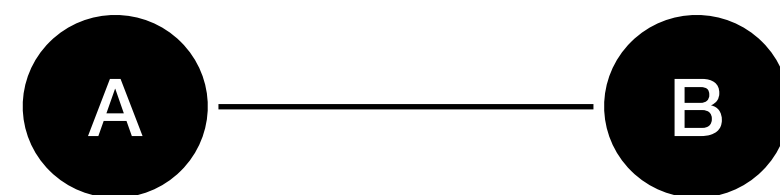
Què és un graf?

- El graf és una estructura de dades **NO LINEAL**

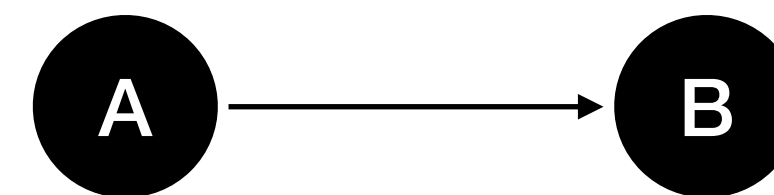


Què és un graf?

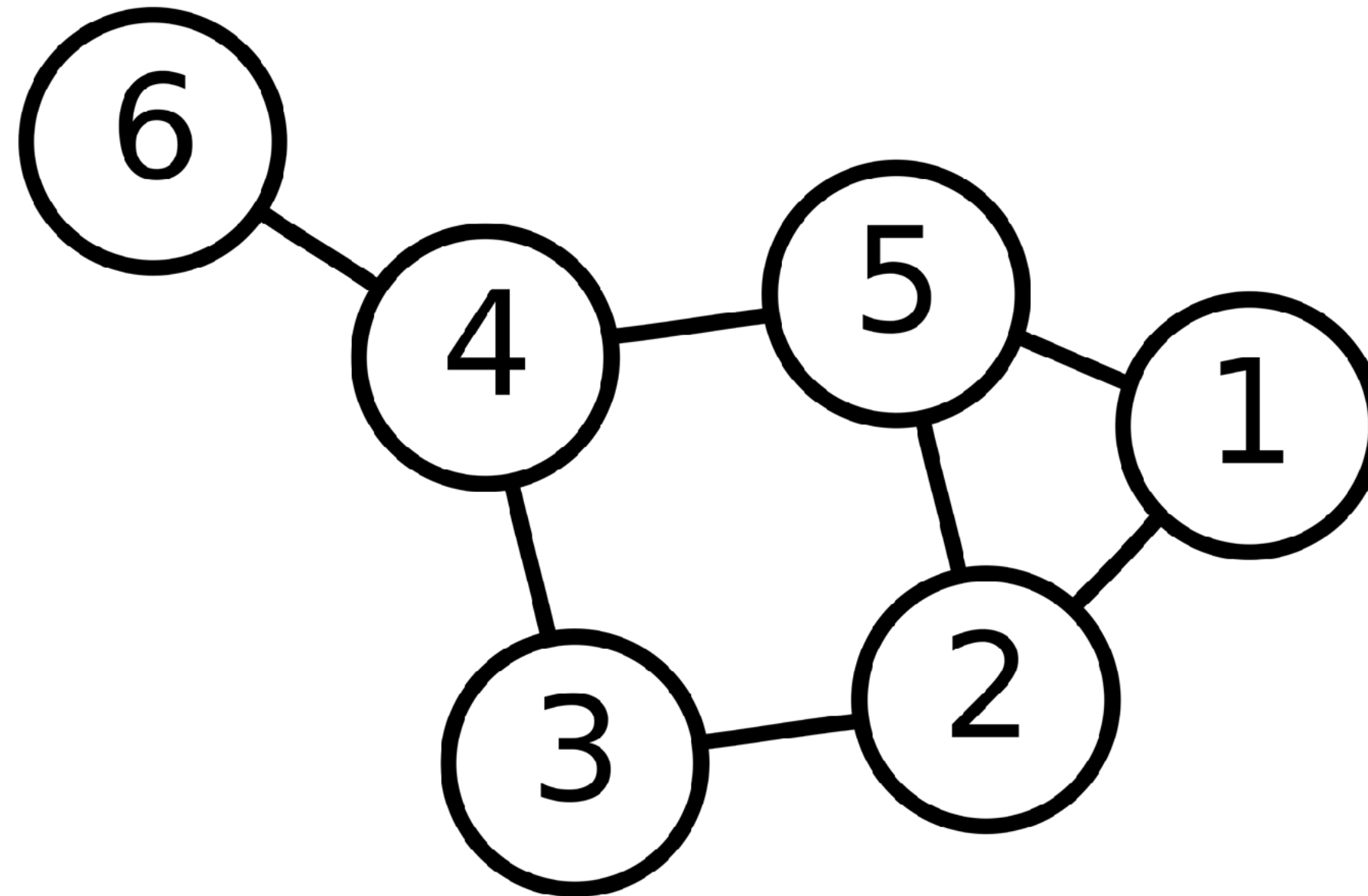
- Un **graf** és un conjunt V de vèrtexs o nodes i un conjunt E de d'arestes o arcs relacionats entre aquests vèrtexs. Els grafs tenen una estructura topològica que ens permet establir relacions espacials entre els vèrtexs que estem estudiant.
- Un **graf** és un conjunt ordenats $G = (V, E)$ on:
 - V és el conjunt de vèrtex, i
 - E és un conjunt d'arestes que relacionen aquests vèrtexs

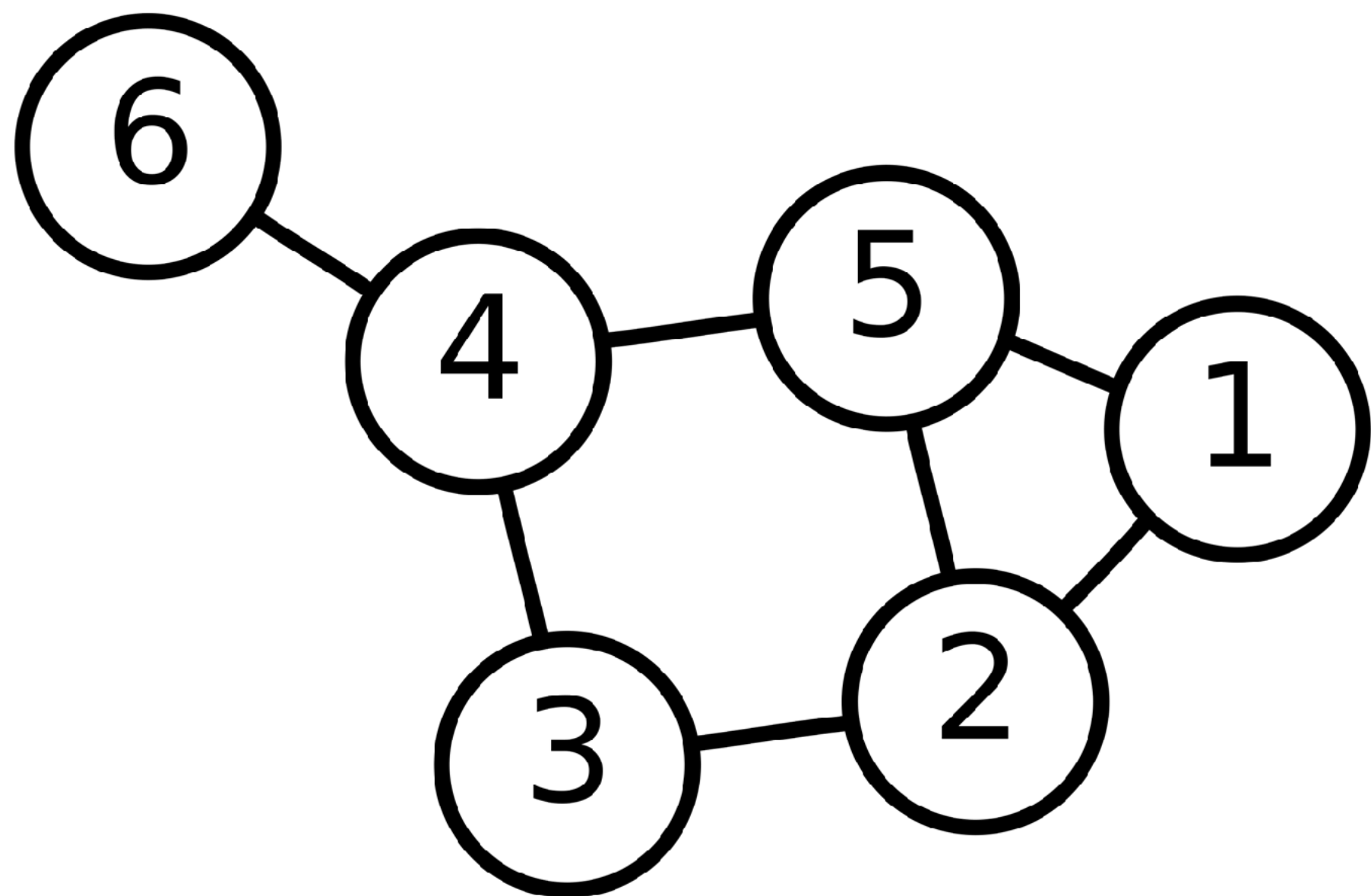


Graf no dirigit



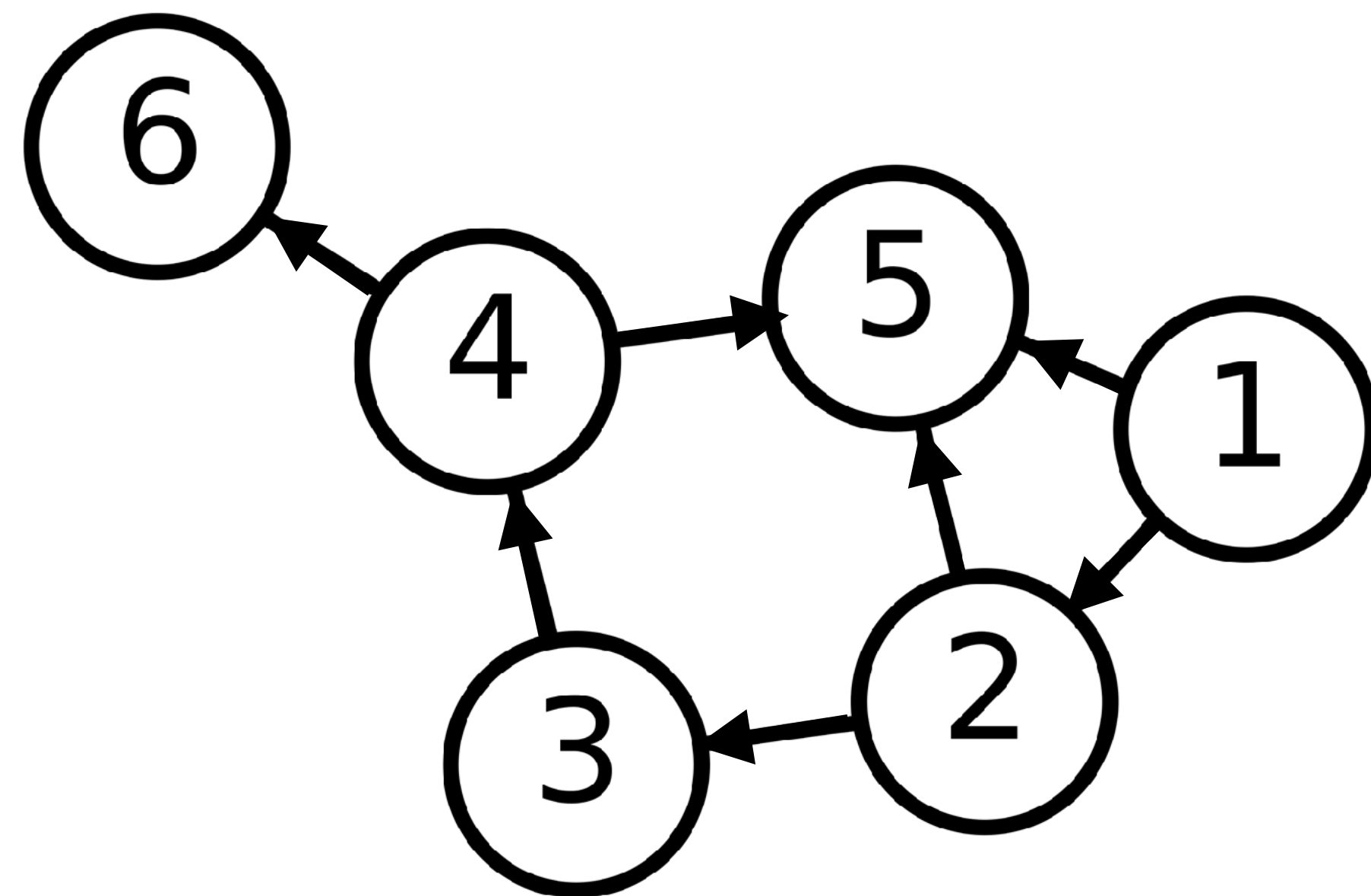
graf dirigit





$$V = \{1,2,3,4,5,6\}$$

$$E = \{\{1,2\}, \{1,5\}, \{2,3\}, \{2,5\}, \{3,4\}, \{4,5\}, \{4,6\}\}$$

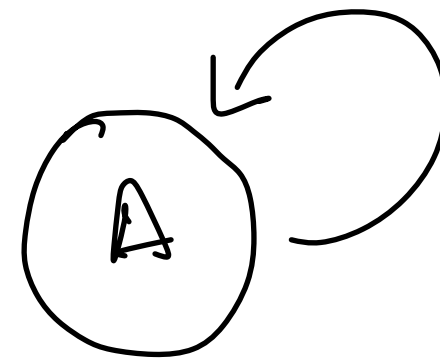


$$V = \{1,2,3,4,5,6\}$$

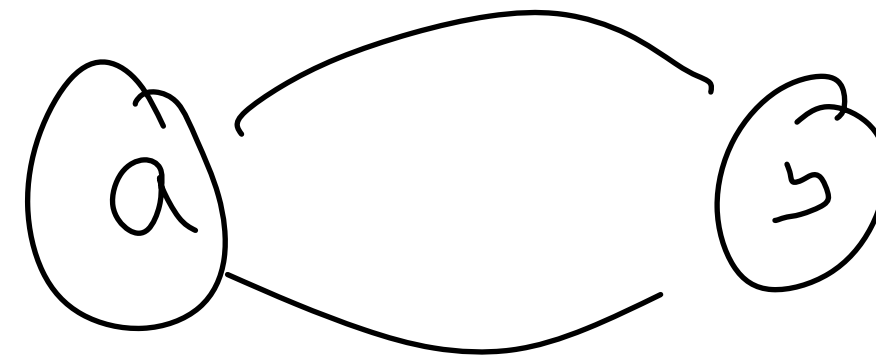
$$E = \{(1,2), (1,5), (2,3), (2,5), (3,4), (4,5), (4,6)\}$$

Graf simple

- Tenim un bucle (o **loop**) si tenim un node connectat amb si mateix, i.e. tenim una aresta que comença i acaba al mateix node.



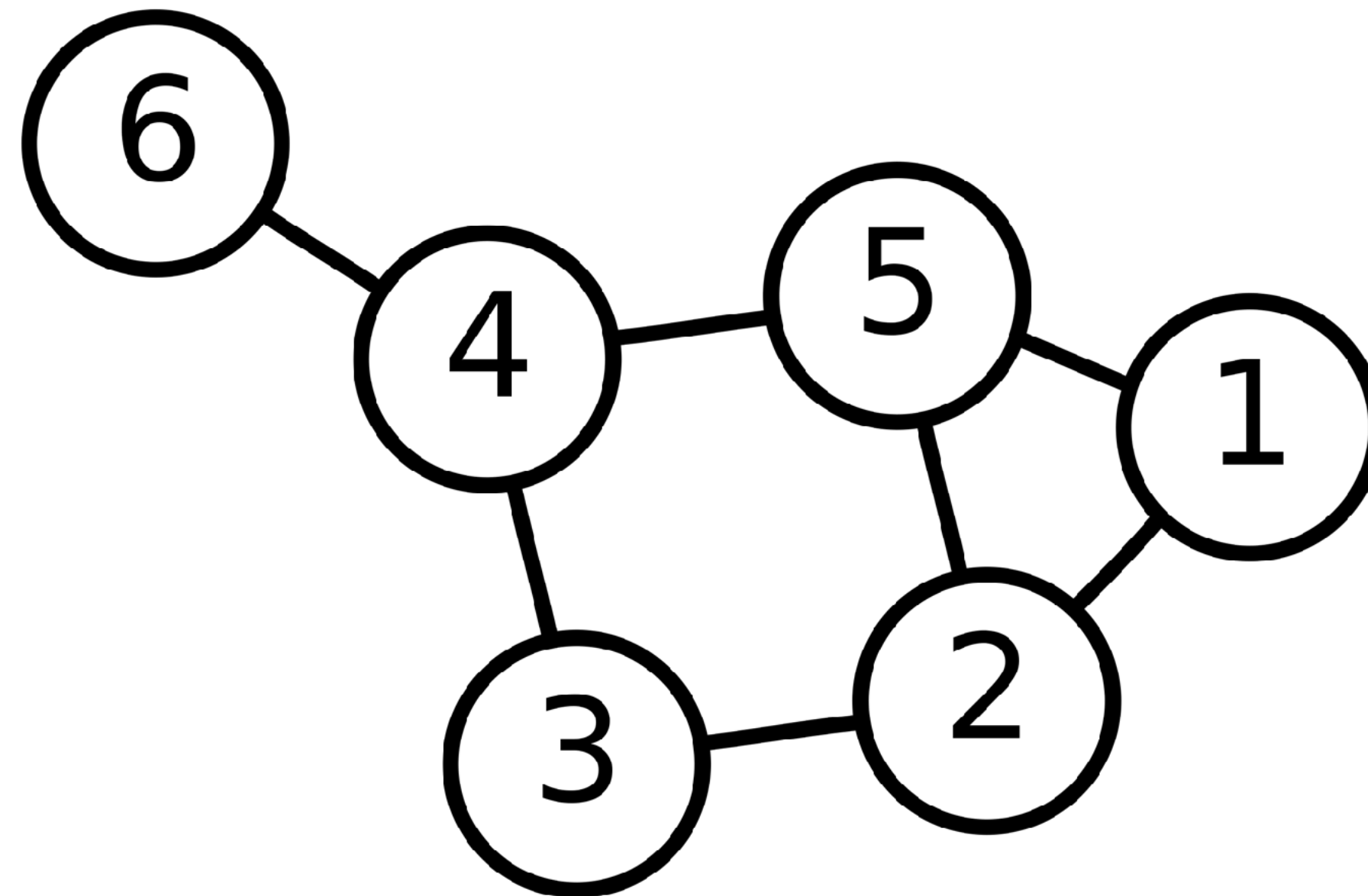
- Múltiples arestes entre dos nodes



- Diem que el graf es **simple** si el graf és no dirigit, no té cap bucle i com a màxim una aresta entre qualsevol parella de vèrtexs diferents.

És això un graf simple?

Text del títol

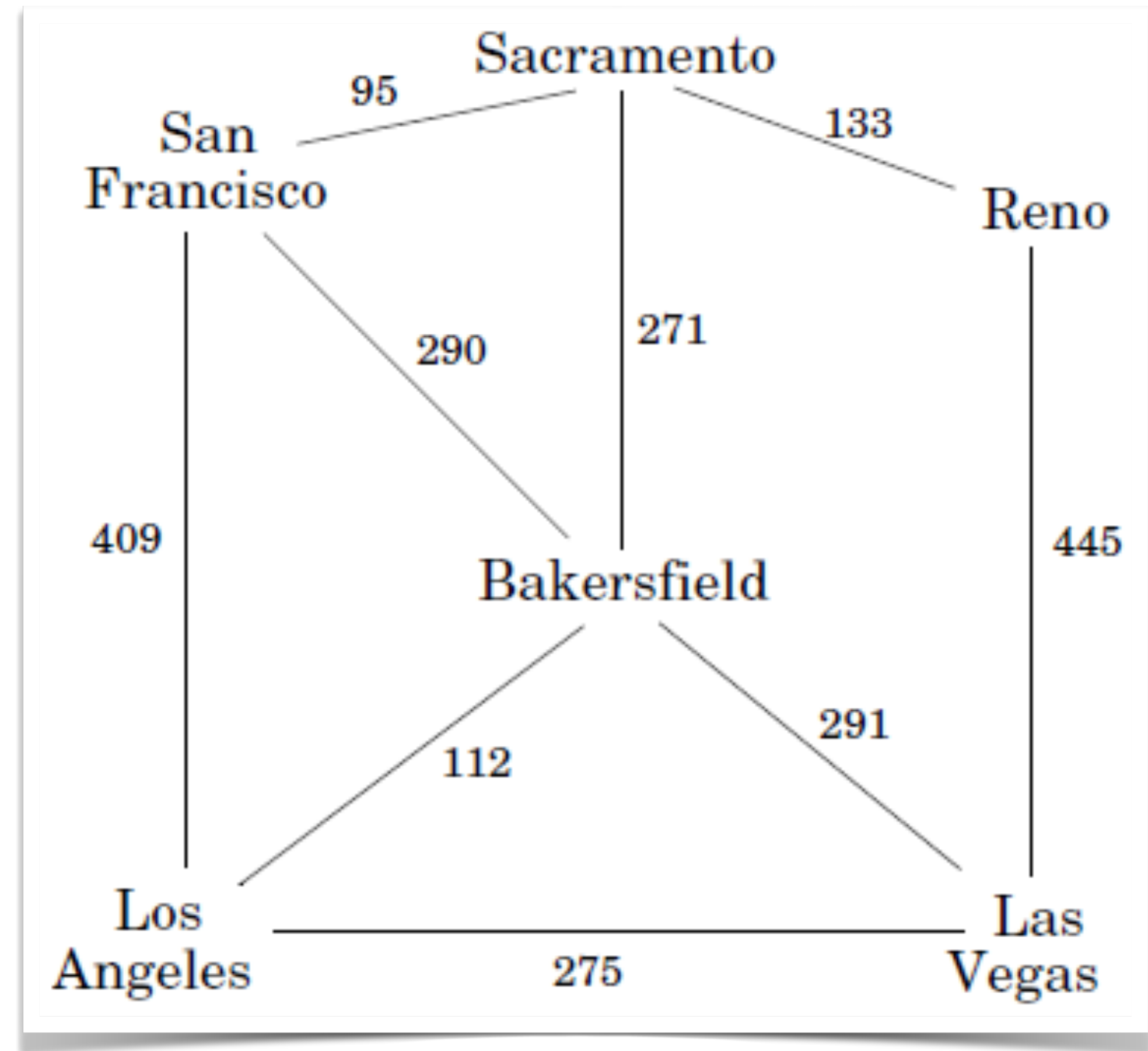


Grafs amb pesos

Algoritmes sobre grafos

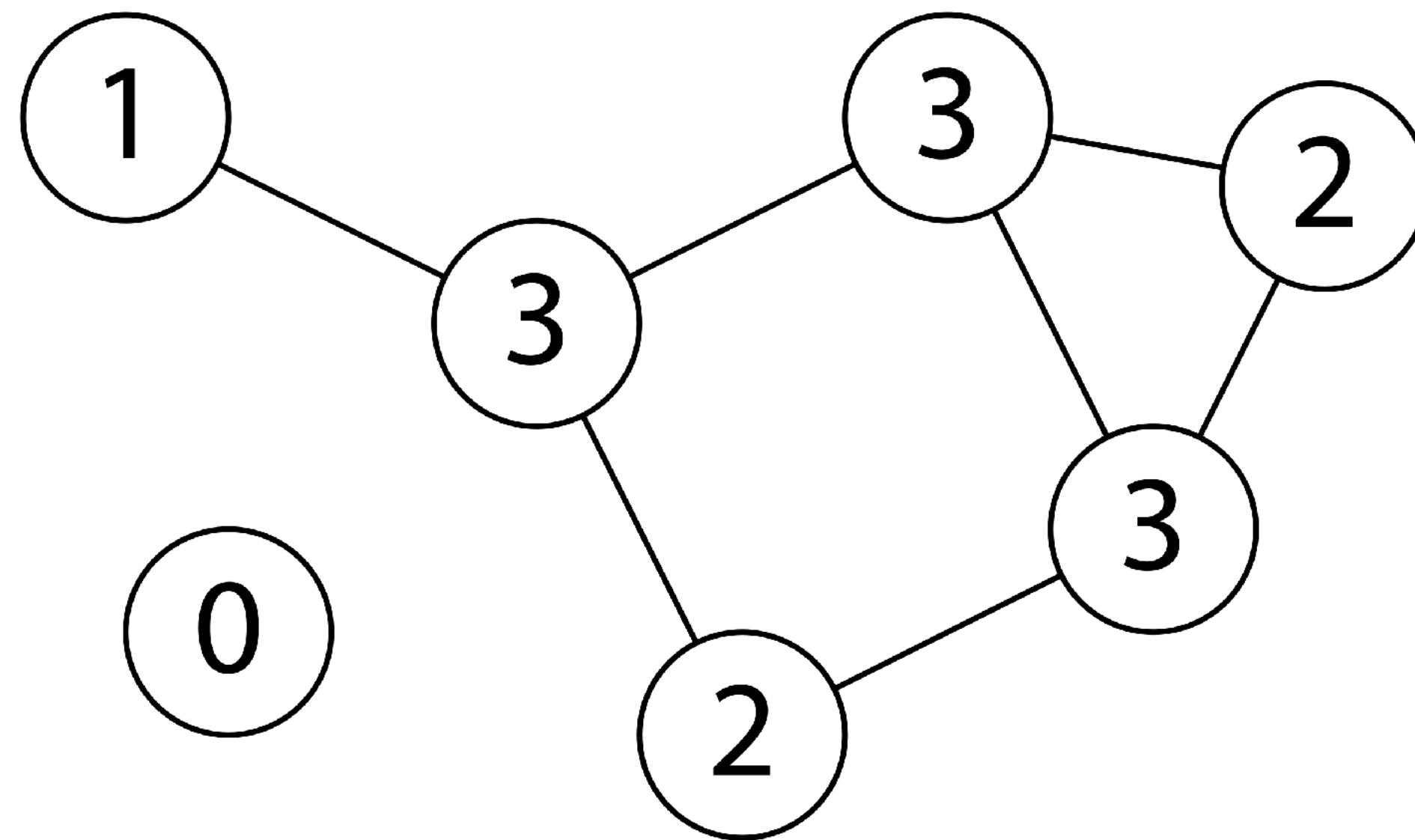
- **Pesos a les arestes**

Exemple amb distàncies



Propietats dels grafos

- El **grau** d'un **vèrtex** és el nombre d'**arestes** que hi incideixen

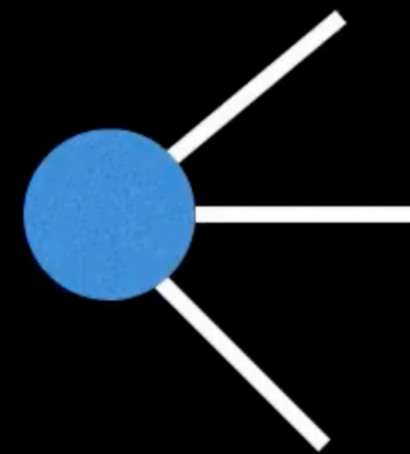


Un graf amb vèrtexs etiquetats segons el seu grau. El *vèrtex aïllat* s'etiqueta amb 0, ja que no és adjacent a cap altre vèrtex.

Què és el grau d'un node?

Text del títol

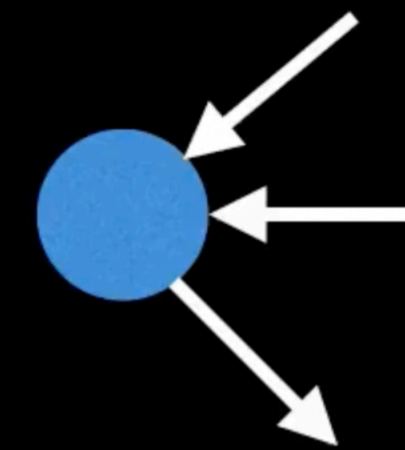
Graf no dirigit



Grau (degree) del node = 3

El grau d'un node és el
nombre d'arestes incidents a
ell

Graf dirigit



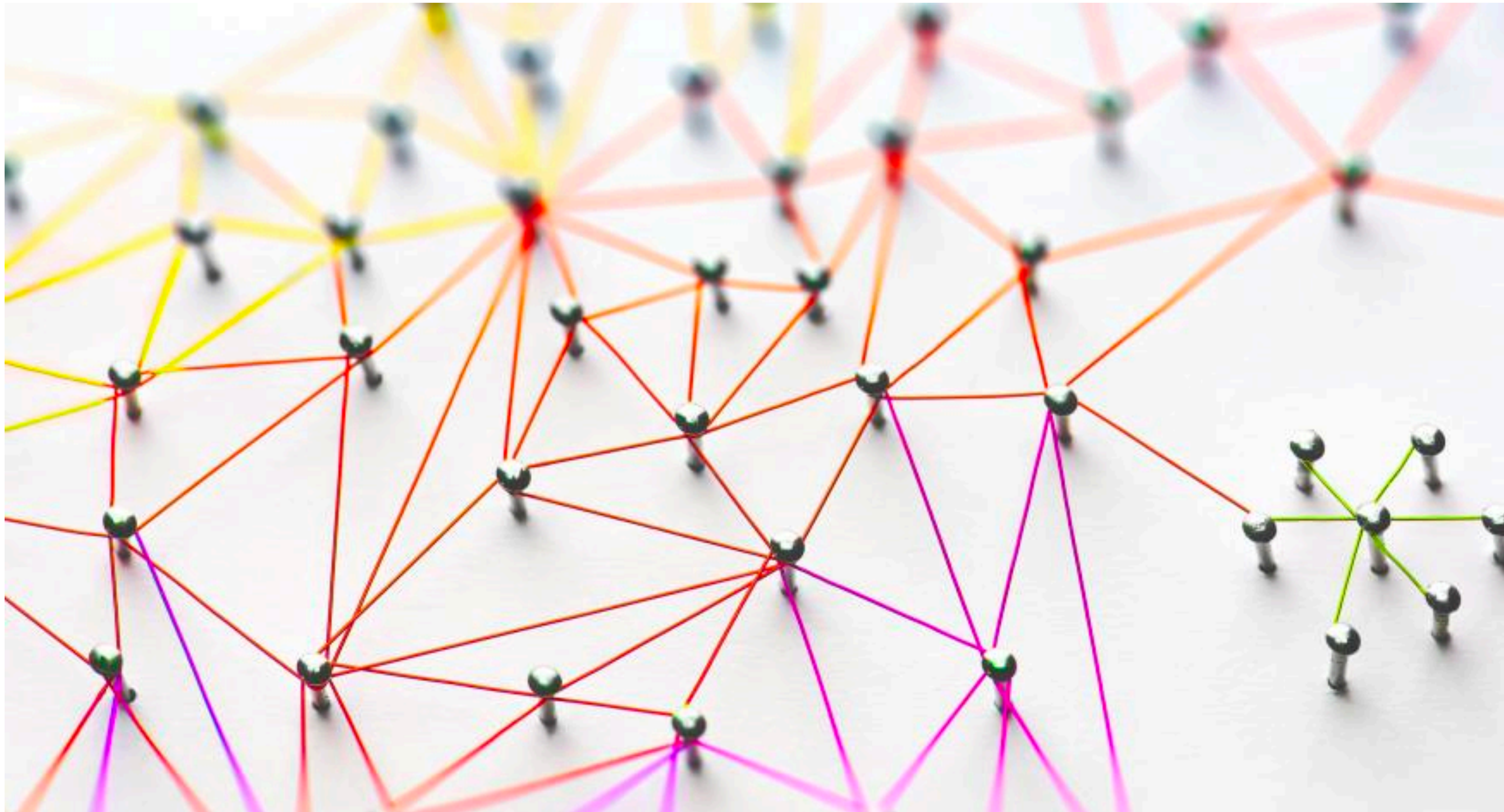
Grau d'entrada (In degree) = 2

Grau de sortida (Out degree) = 1

El grau d'entrada (indegree) és el
nombre d'arestes d'entrada i el
grau de sortida (outdegree) és el
nombre d'arestes de sortida

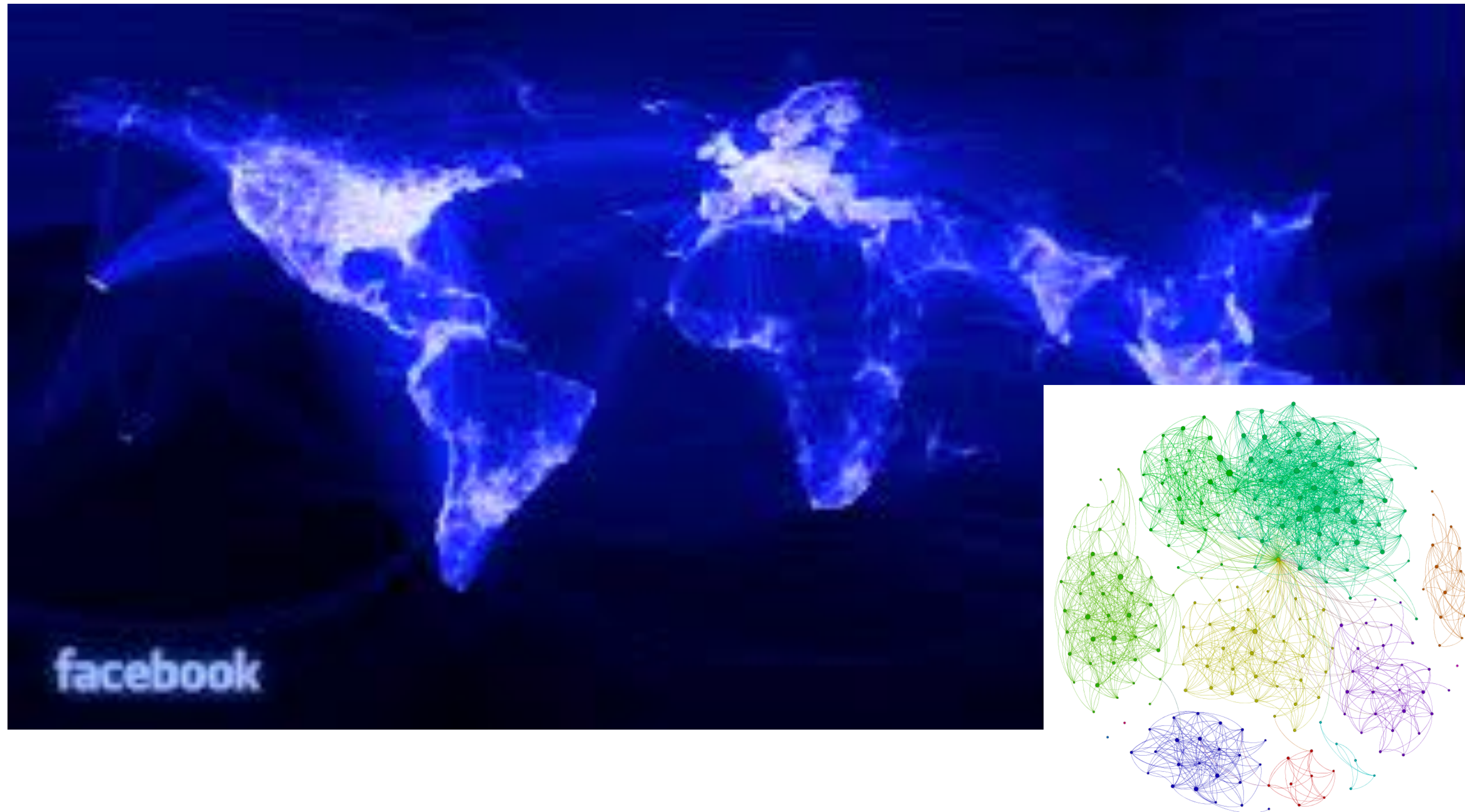
Exemples

Principalment treballarem amb grafs



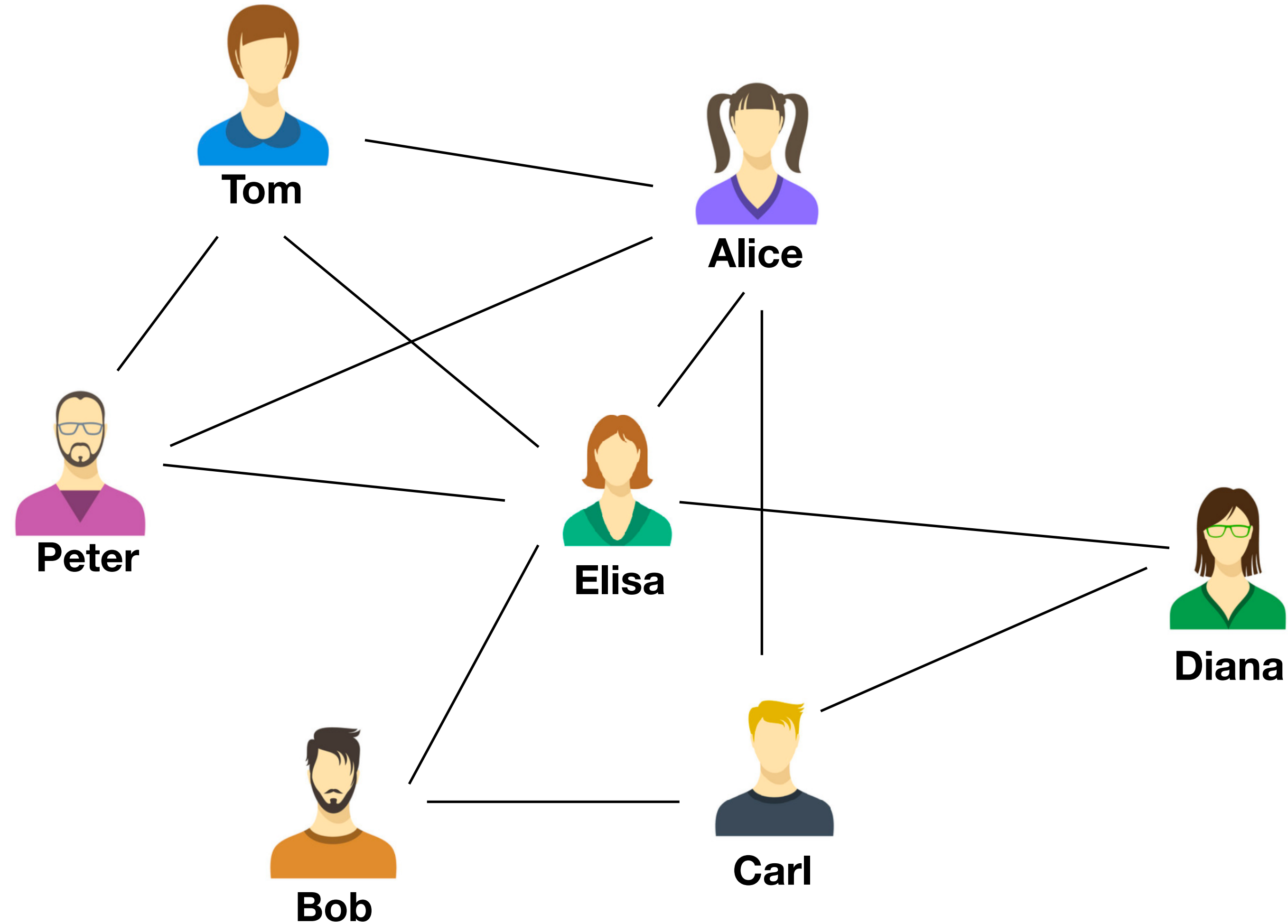
Xarxes socials

Text del títol



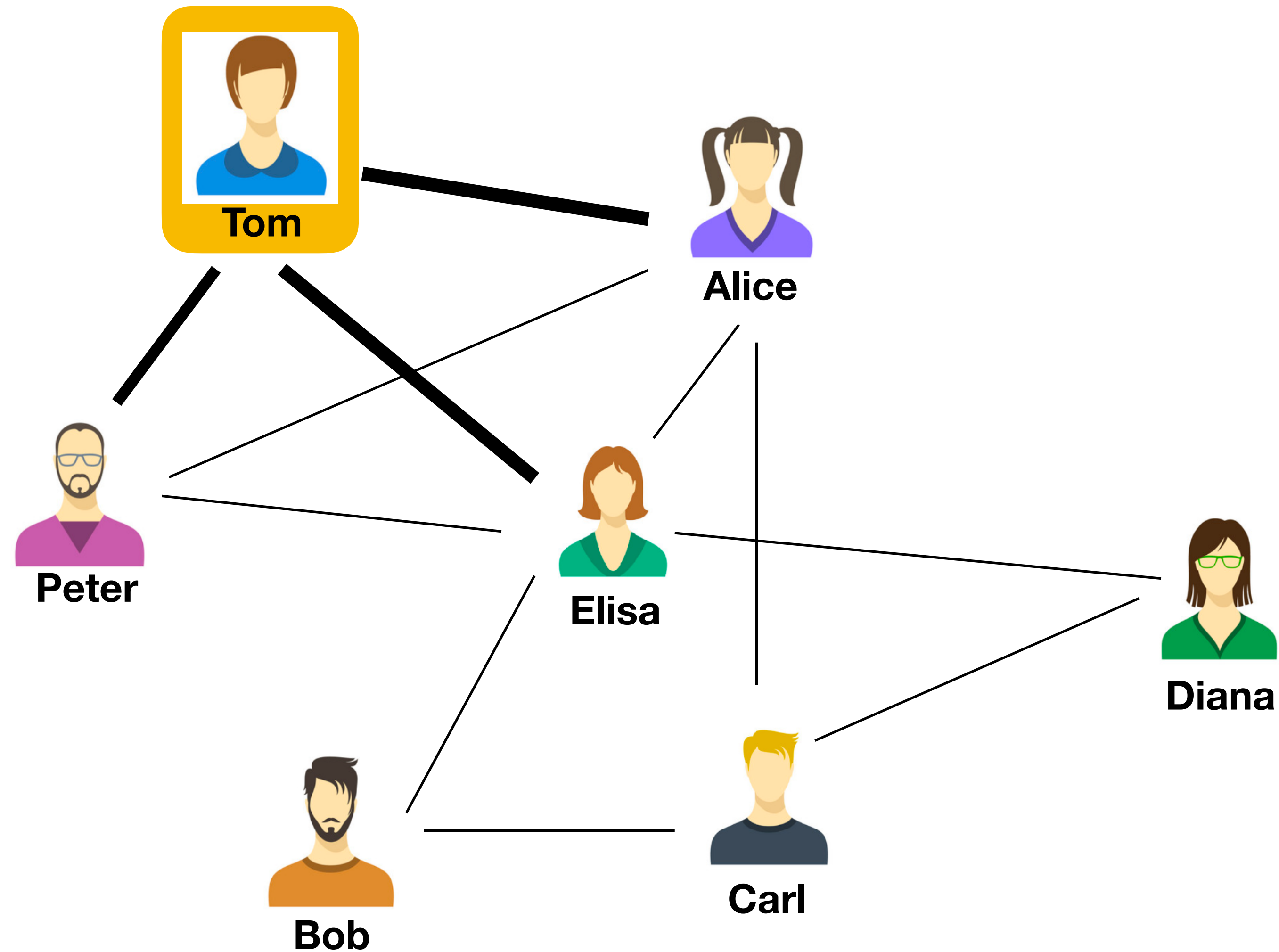
Social Networks

Text del títol



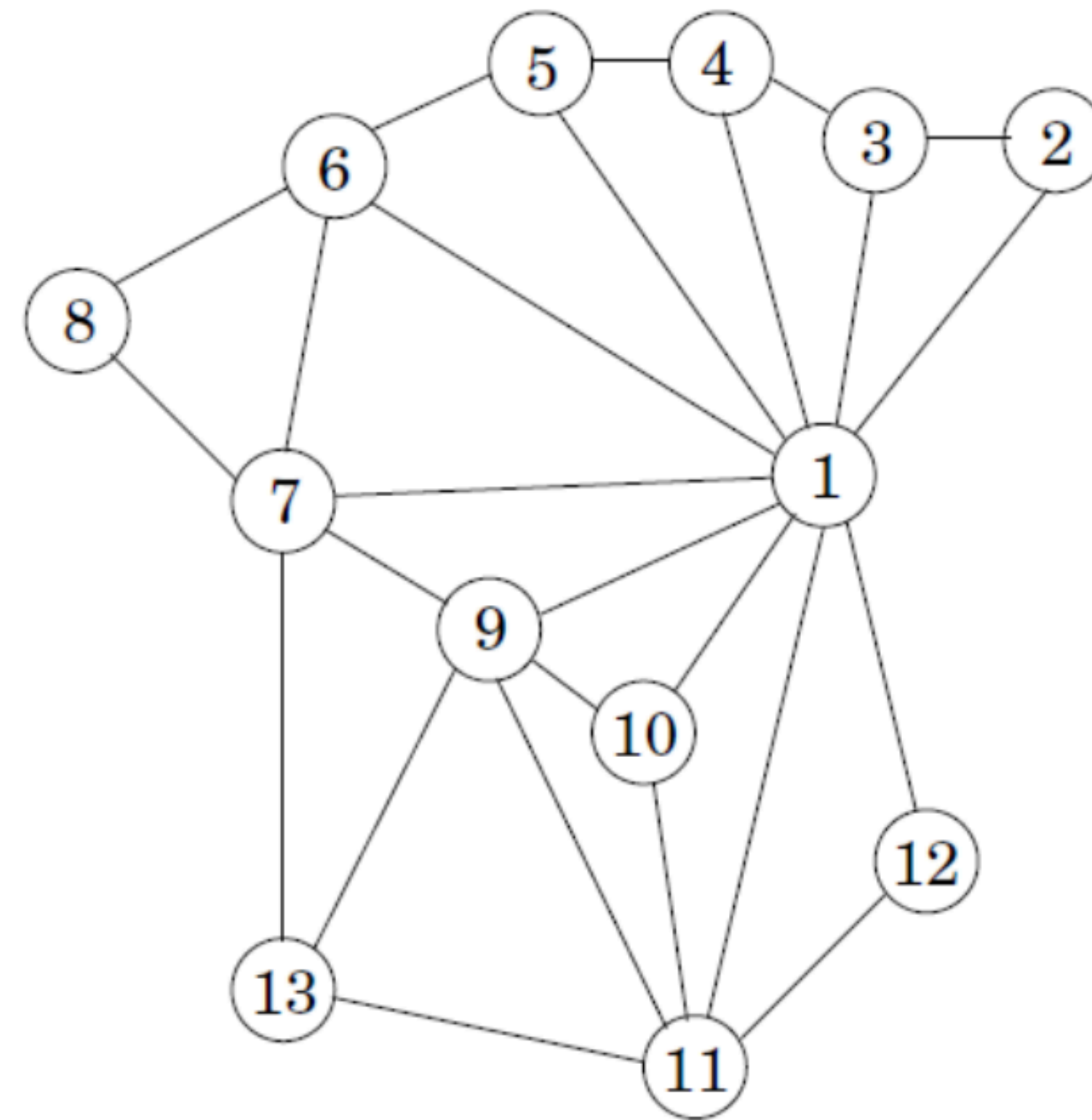
Social Networks

Podem suggerir algun contacte al Tom?



Navegació

Optimització de rutes

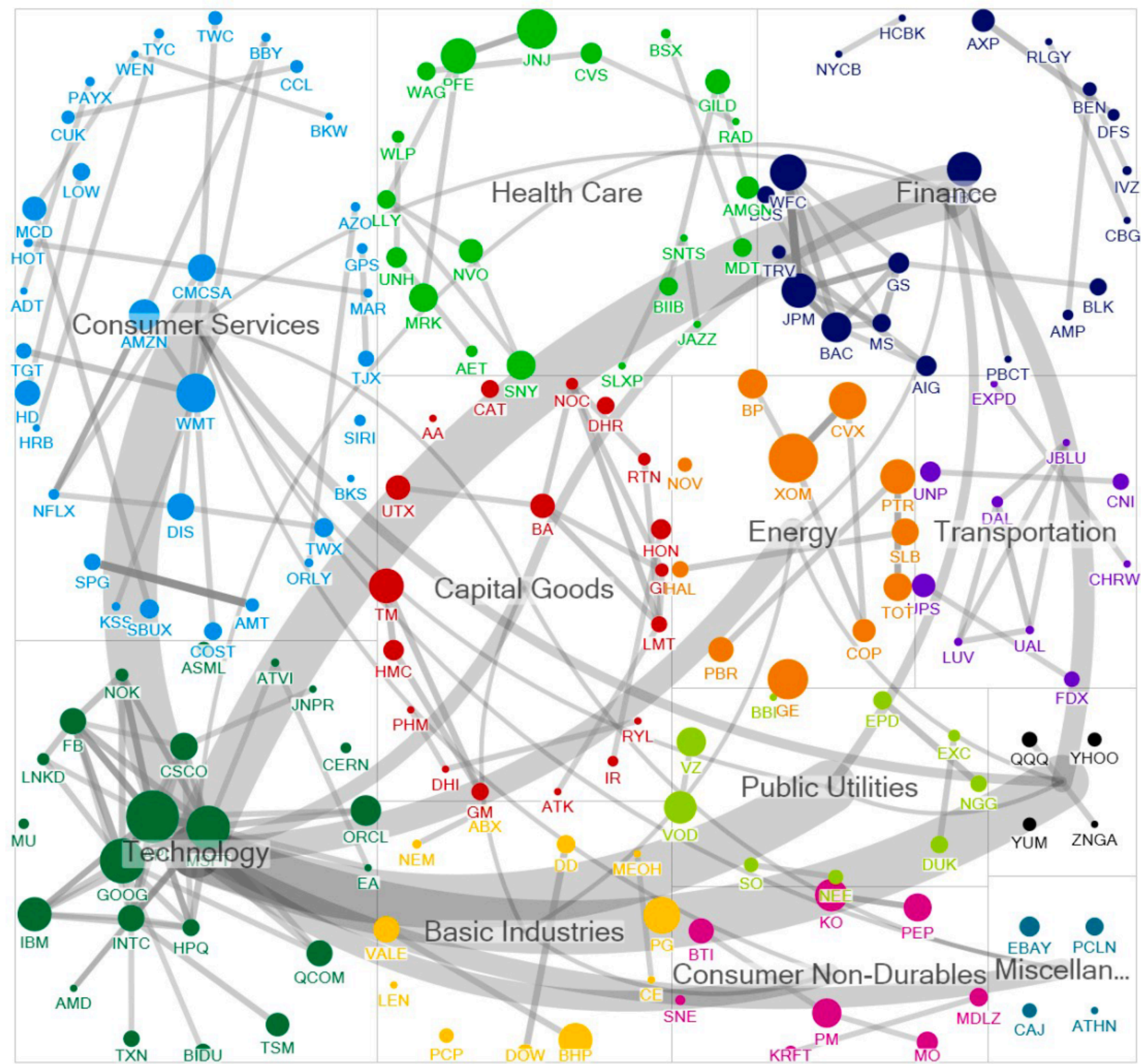


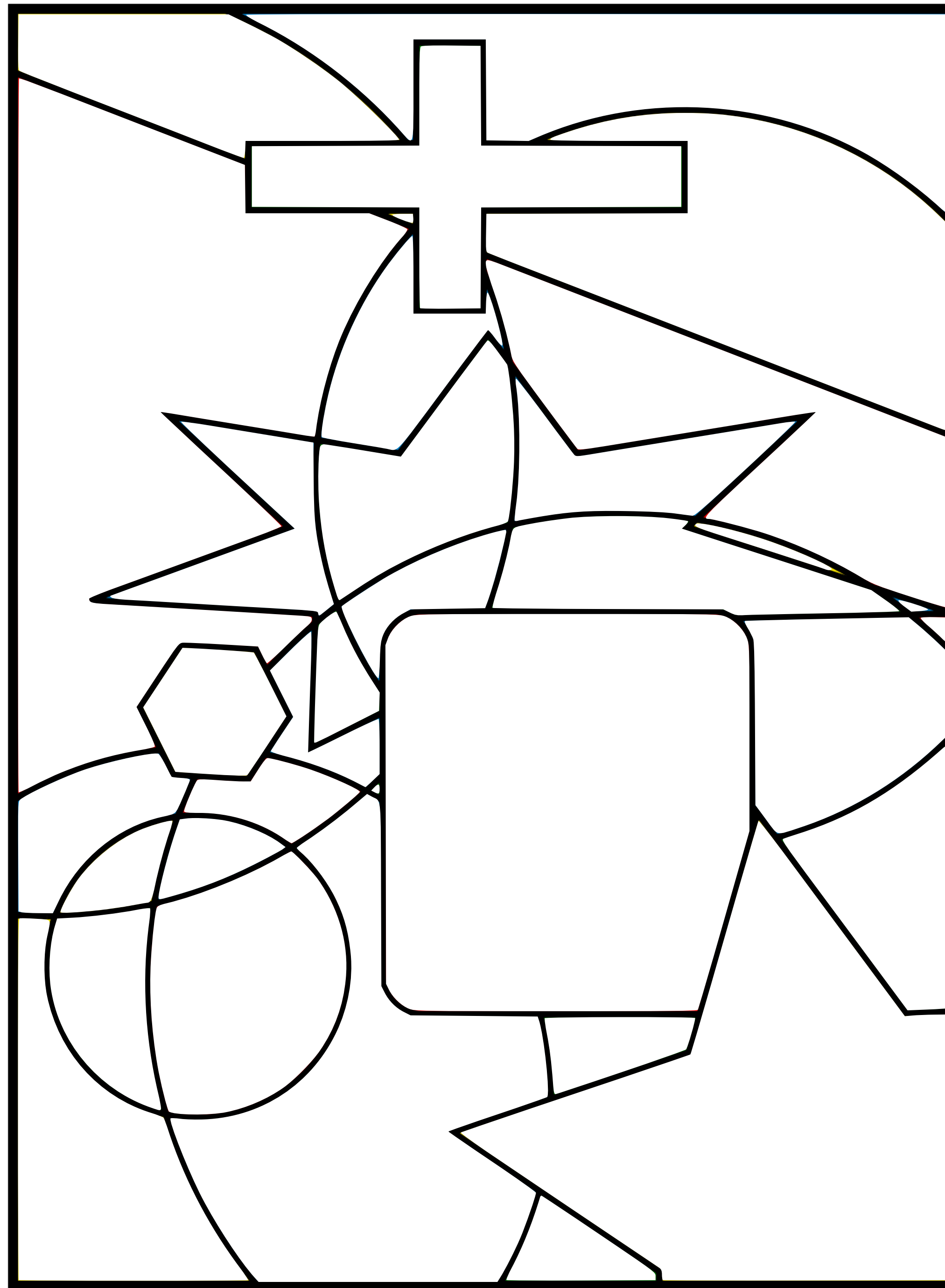
Visualitzacions y generació de rutes

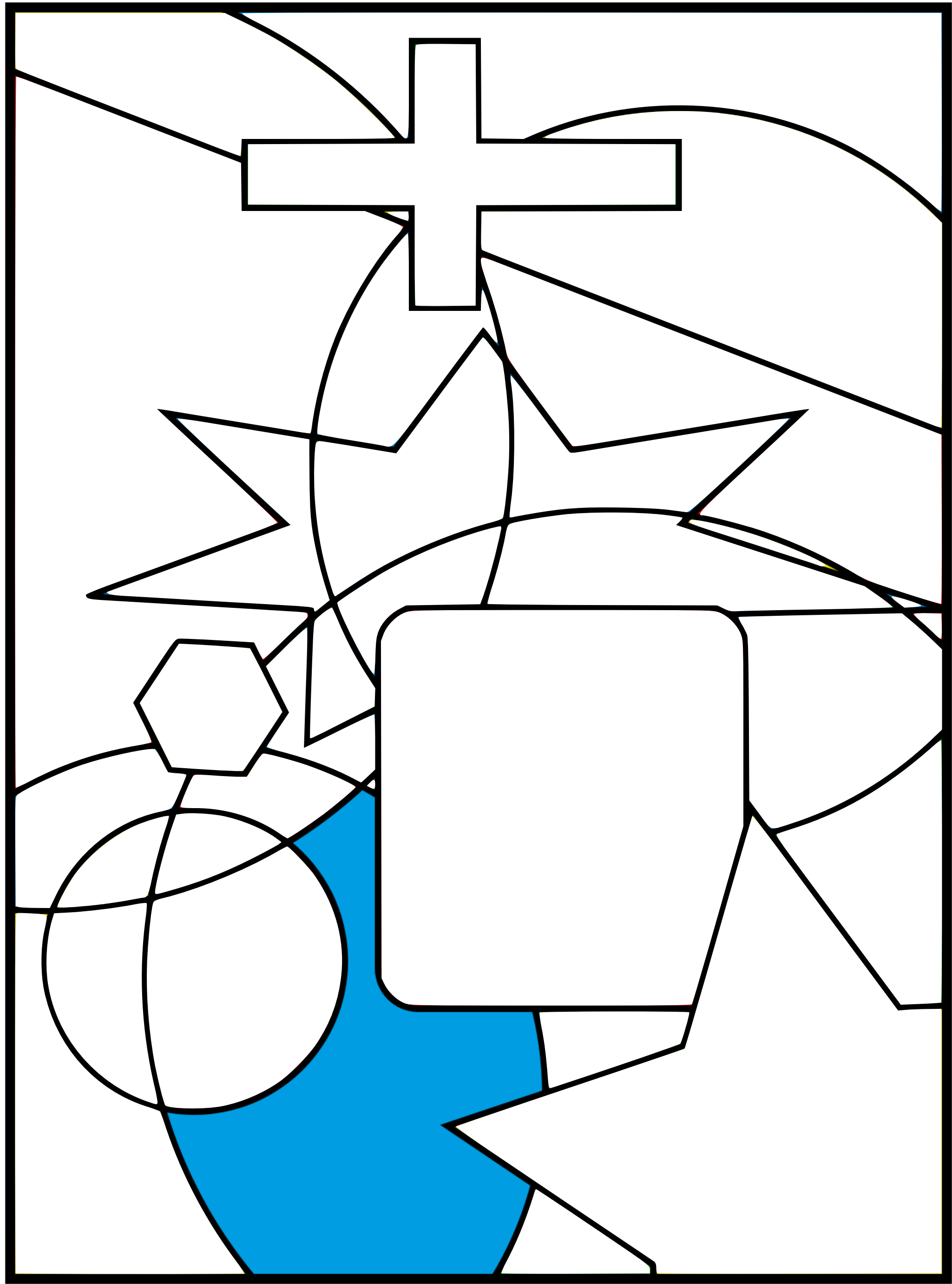


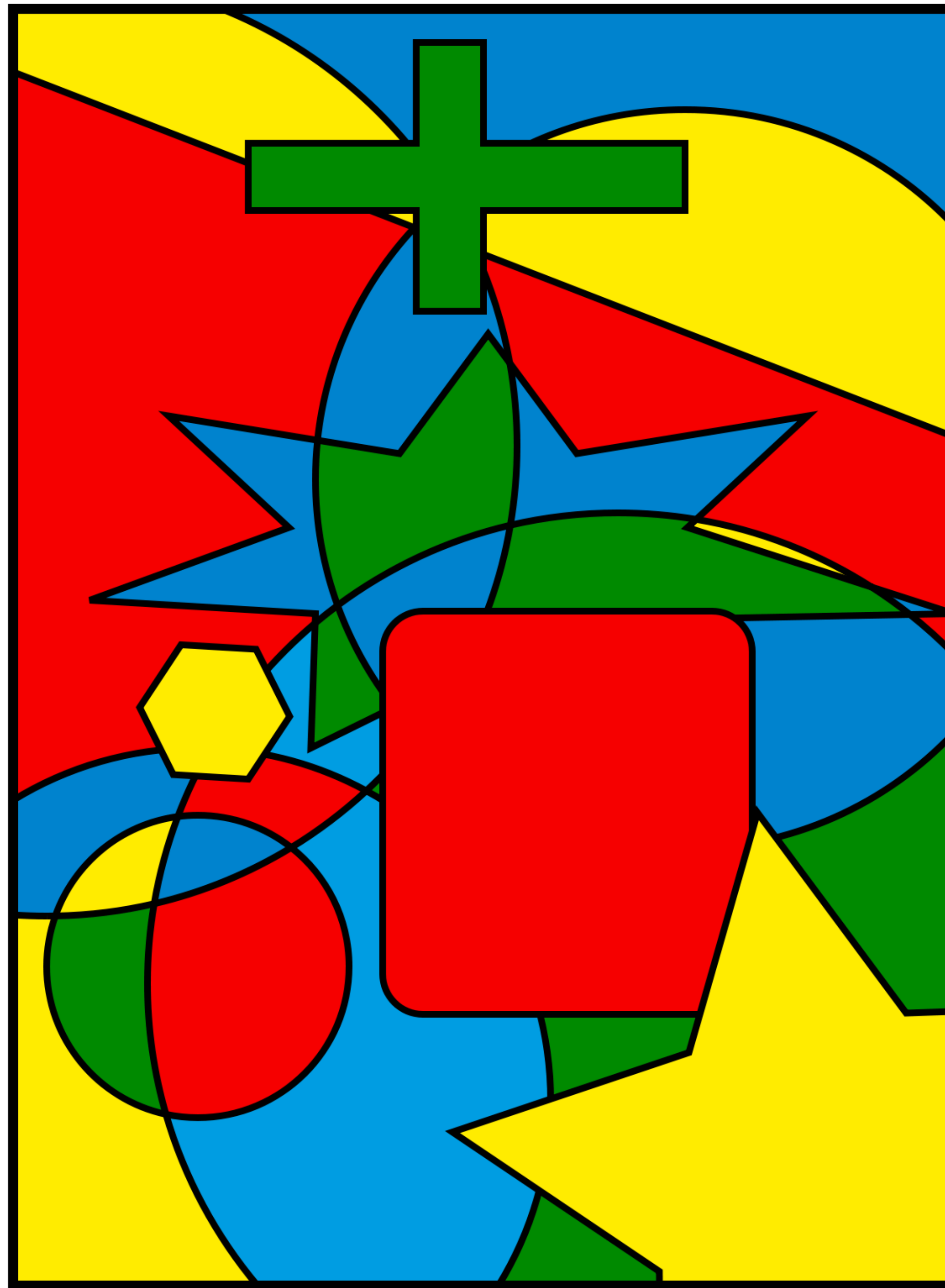
Com podem visitar tots els PDIs fent el menor número de quilòmetres?



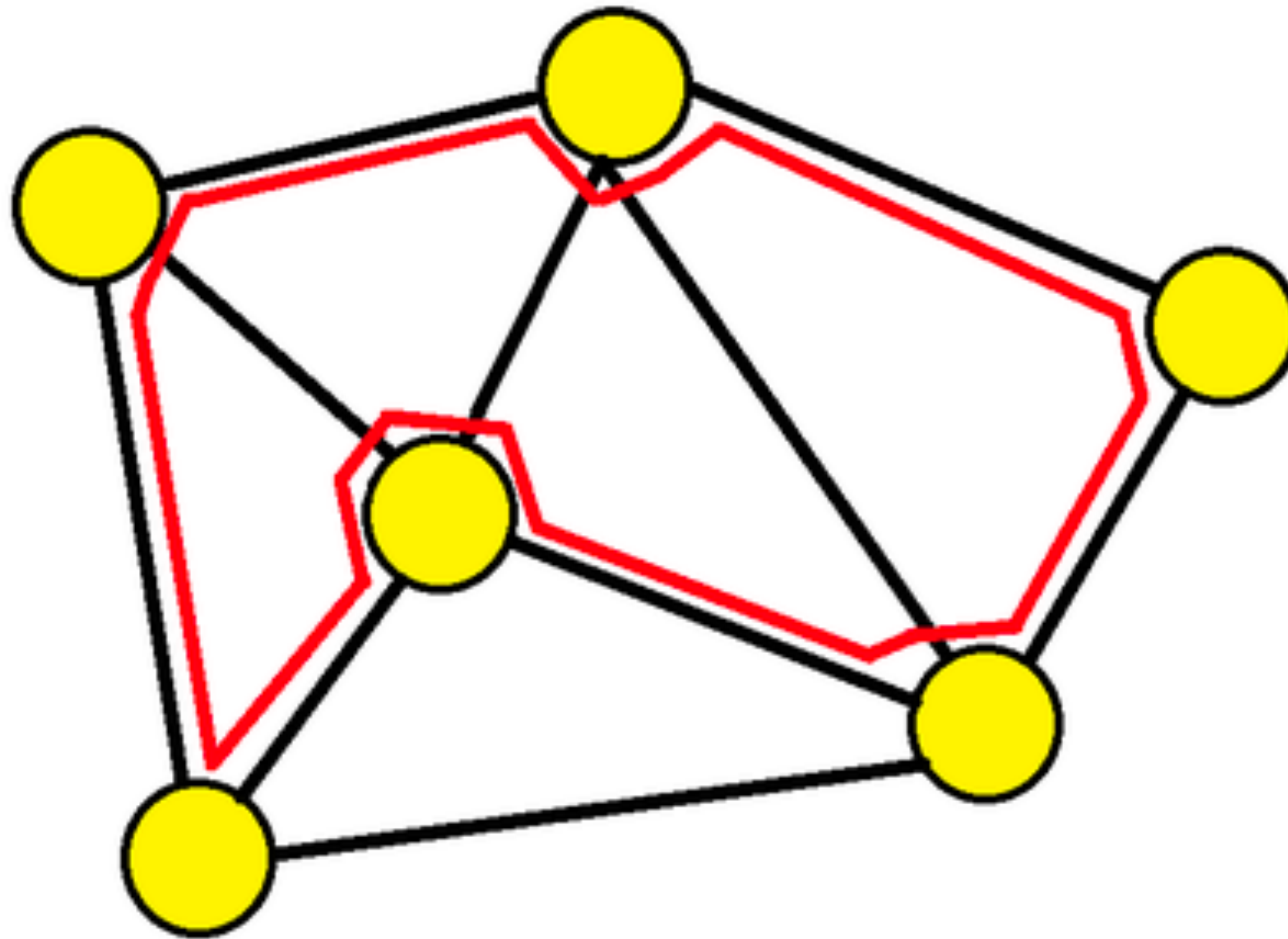


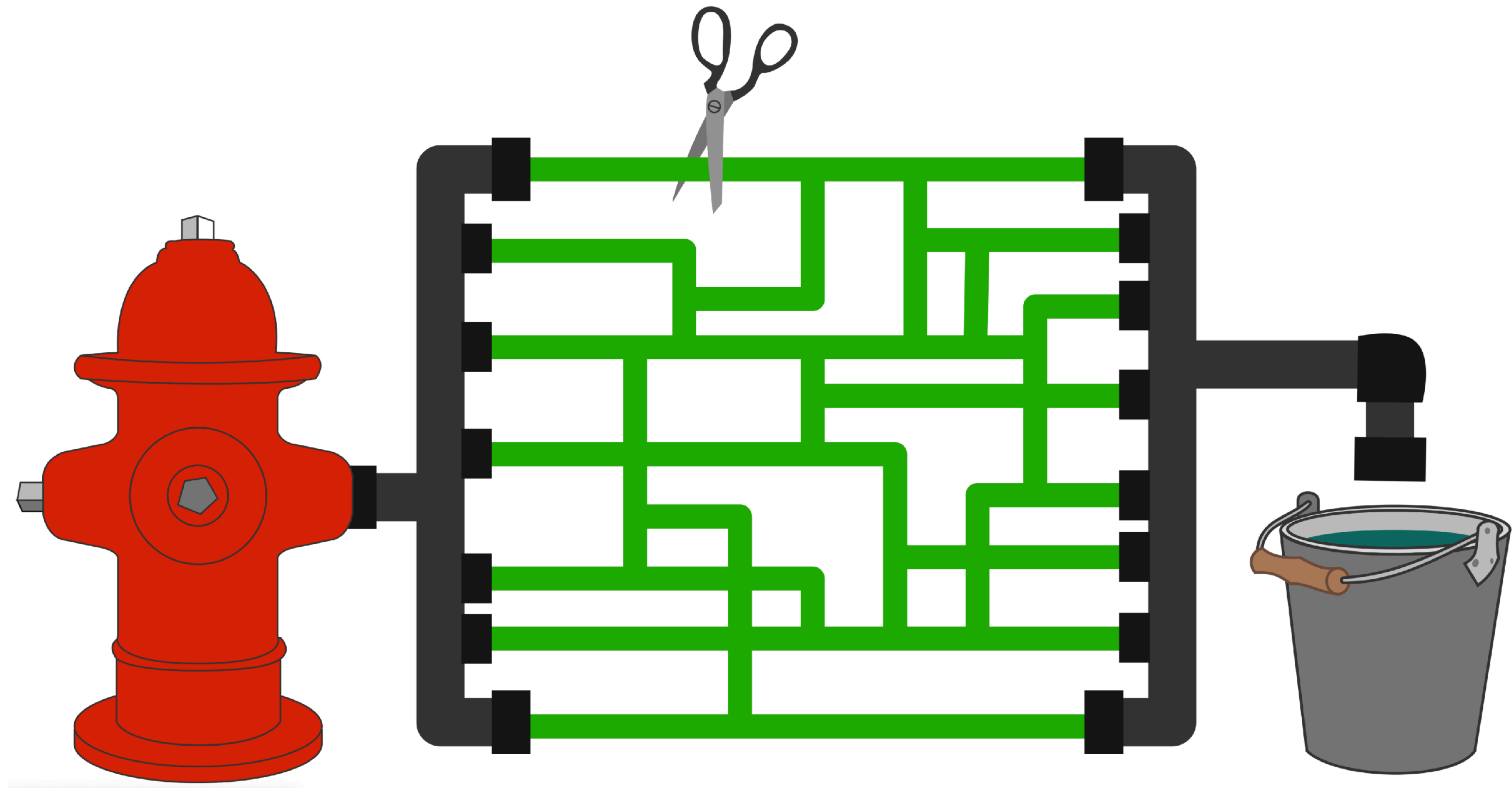






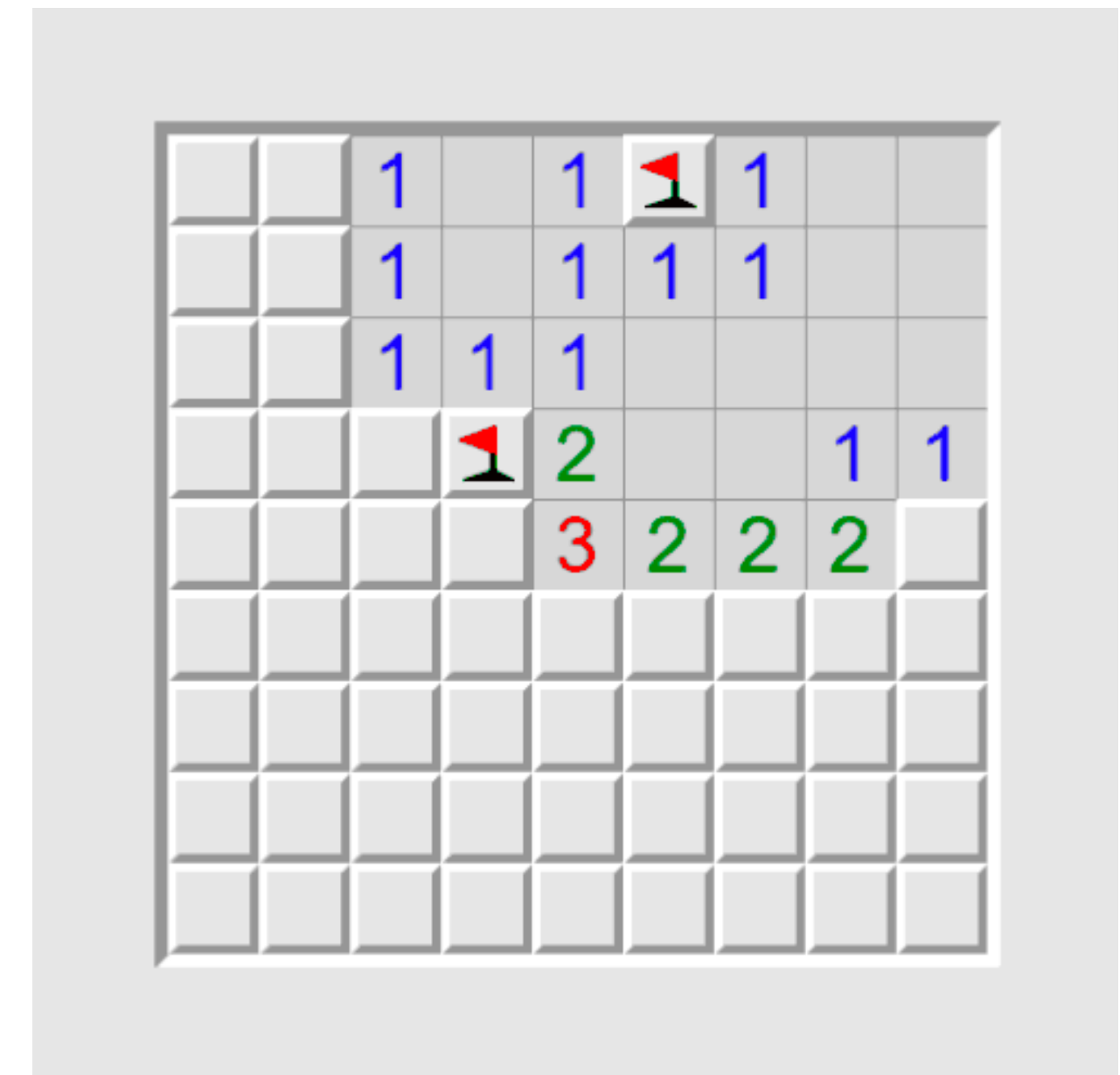
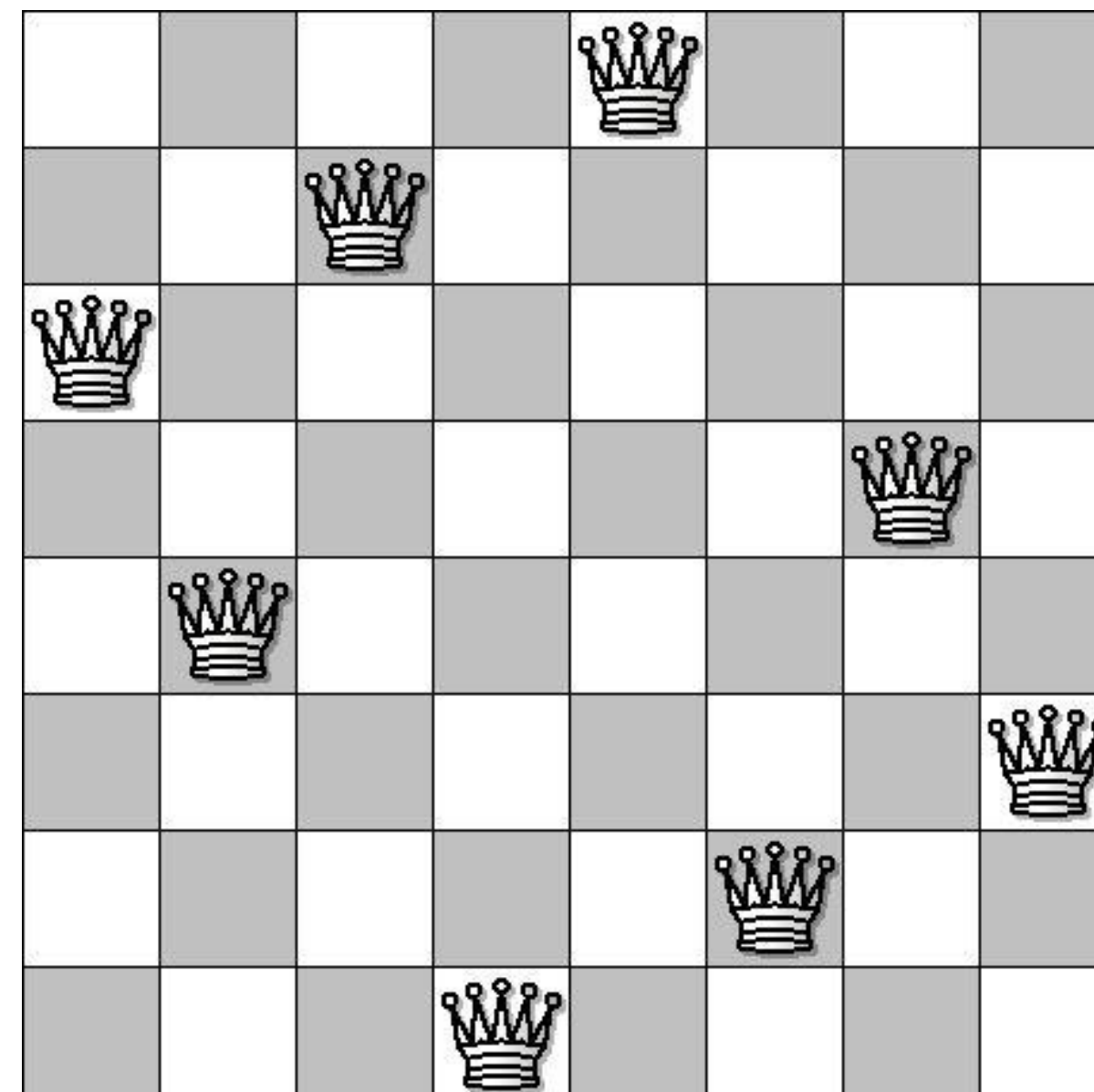
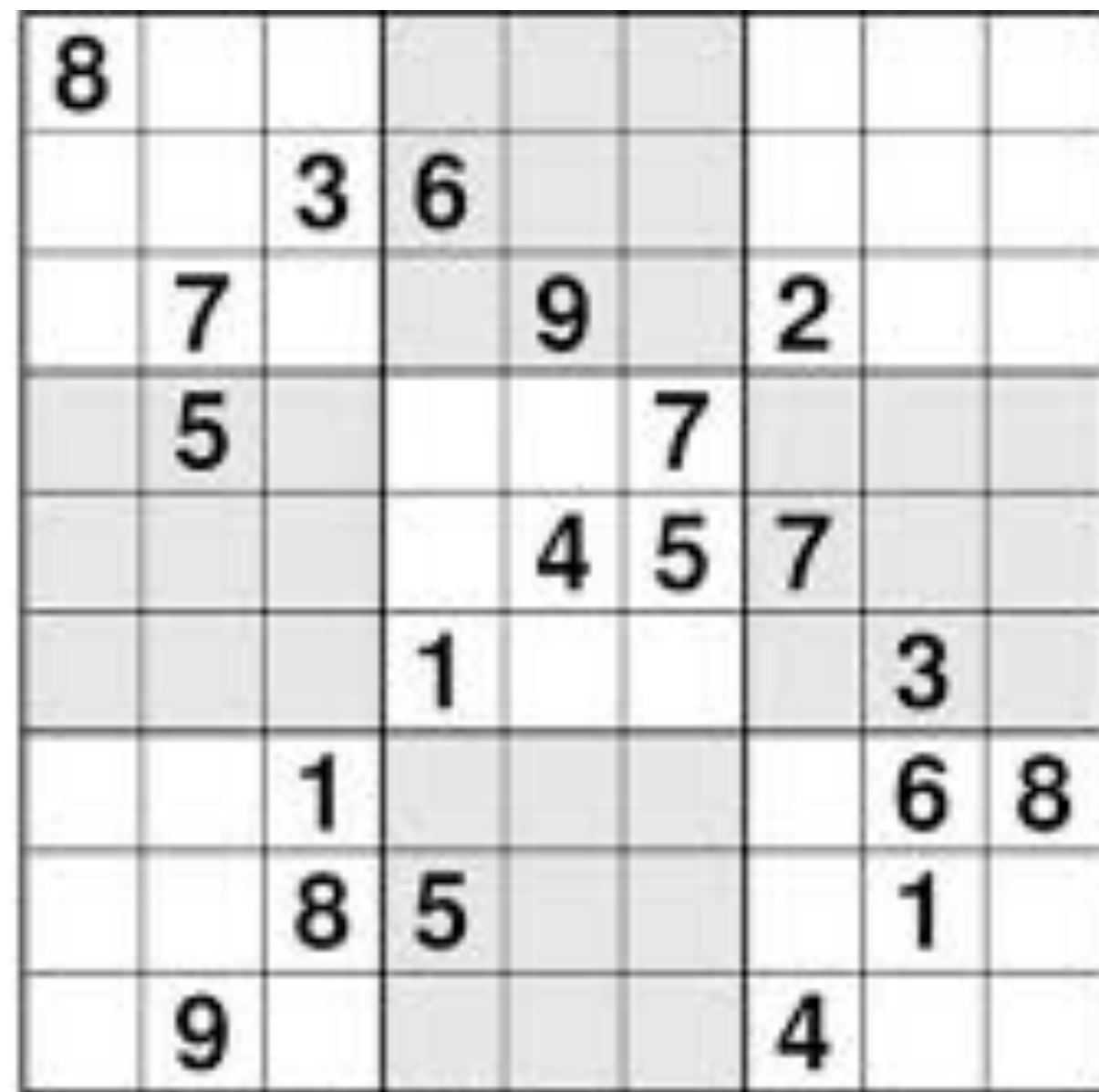
Hi ha un cicle que estrictament ens permet passar un cop per cada node?





Jocs

Estratègies basades en grafs



Aplicacions amb grafs

Graph	Vertex	Edge
Network Communication	telephone, computer	fiber optic cable
circuit	gate, register, processor	wire
financial	stock, currency	transactions
transportation	street intersection, airport	highway, airway route
game	board position	legal move
social relationship	person, actor	friendship, movie cast
molecule	atom	bond
neural network	neuron	synapse

Algunes preguntes interessants!

- **Camí (Path):** Hi ha cap camí entre els nodes s i t ?
- **Camí més curt:** Quin és el camí més curt entre els nodes s i t ?
- **Cicle:** Hi ha cap cicle en el graf?
- **Camí d'Euler:** Hi ha camí que em permeti visitar cada node passant exactament un cop per cada aresta?
- **Camí Hamiltonià:** Hi ha un camí en el graf que passa exactament un cop per cada vèrtex?
- **Connectivitat.** Hi ha cap manera de connectar tots els nodes
- **MST.** Quina és la millor manera de connectar tots els nodes d'un graf?
- **Bioconnectivitat.** Hi ha cap node que la seva eliminació desconnecti el graf.
- **Planaritat.** Podeu dibuixar el gràf en el pla sense vores de creuament

Quins d'aquests problemes són fàcils? difícil? intractable?

Planificació Organització

Algorísmica Avançada

Text del títol

Grafs

Què són els grafs?
Com es representen?
DFS /BFS/ Component Connexes
Com trobar els camins més curts entre dos nodes (Dijkstra/
BellmanFord)
Problema Flux Màxim
Circuits i camins eulerians

Algorítmes "greedy"

Un **algoritme "greedy"** és un algoritme que, per resoldre un problema d'optimització, fa una seqüència d'eleccions, prenent en cada pas un òptim local, amb l'esperança - no sempre complerta - d'assolir una solució òptima global. L'algoritme voraç no torna mai enrere per re-avaluar les eleccions ja preses.
Kruskal / Prim
Problema de la Motxilla/Viatger de Comerç

Programació Dinàmica

Problemes d'optimització de tasques per tal d'arribar a una solució òptima
Floyd Wharshall
Problema de la Motxilla, Levenshtein
Ponts de Hanoi
...

Enumeratius

Backtracking, Ramificació i Poda
Sudoku, 8-Reines,
Problema de la Motxilla
...
...

Objectius del curs

- Anàlisi i disseny avançat d'algoritmes, tant des d'un punt de vista teòric com aplicat

Què esperem dels estudiants?

- Els estudiants han de participar de forma activa durant les classes de teoria (1.5 hores a la setmana). Durant les hores presencials de pràctiques i problemes (1.5 i 1.5 hores cada setmana excepte les setmanes marcades com NO classe) hauran de resoldre de forma individual una sèrie de problemes. Les hores no presencials de l'assignatura (4 hores a la setmana) les han de dedicar a l'estudi de la teoria i a la preparació de les pràctiques.

Contactes:

Teoria Online

- **Santi Seguí**

Email: santi.segui@ub.edu

Sessions Problemes

- **Manuel Andrés Hernández**

Email: b.miquel@ub.edu

Laboratoris

- **Santi Seguí**
- **Sergio Escalera**
- **Pablo Martínez**

Email: santi.segui@ub.edu

Email: sescalera@ub.edu

Email: pablo.martinez@ub.edu

- **Com seran les classes Teòriques? (1.5 hora la setmana)**
 - S'introdueixen els conceptes teòrics de l'assignatura.
- **Com seran les classes problemes? (1.5 hores a la setmana)**
 - Es realitzaran amb grups reduïts. Es farà un repàs de la teoria i exercicis associats a les sessions corresponents
 - S'ha de portar portàtil sempre que es pugui!
 - L'alumne ha d'haver treballat el contingut corresponent a la sessió.
- **Com seran les classes de Laboratori? (1.5 hora a la setmana)**
 - S'espera que l'alumne treballi amb les diferents pràctiques de forma autònoma i asíncrona.
 - Les pràctiques **es realitzen amb parelles.**

Programació de l'oferta docent del Primer semestre

Activitat											
Grup	Dies					Horari	Professorat	Llengua	Aula		
Classes teòriques [Presencial]											
M1	dl.	dt.	dc.	dj.	dv.	1r sem.	12.00-13.30	Segui Mesquida, Santiago	Català	Aula B5	
M2	dl.	dt.	dc.	dj.	dv.	1r sem.	13.30-15.00	Segui Mesquida, Santiago	Català	Aula B7	
Pràctiques de problemes [Presencial]											
abc	dl.	dt.	dc.	dj.	dv.	1r sem.	08.00-09.30			Aula B2	
	--						Hernández Alonso, Manuel Andrés	Castellà			
def	dl.	dt.	dc.	dj.	dv.	1r sem.	08.00-09.30			Aula B2	
	--						Hernández Alonso, Manuel Andrés	Castellà			
Pràctiques de laboratori d'ordinadors [Presencial]											
a00	dl.	dt.	dc.	dj.	dv.	1r sem.	09.30-11.00	Martínez Martínez, PABLO	Castellà	Aula IA	
b00	dl.	dt.	dc.	dj.	dv.	1r sem.	09.30-11.00	Pujol Vila, Oriol	Català	Aula IB	
c00	dl.	dt.	dc.	dj.	dv.	1r sem.	11.00-12.30	Segui Mesquida, Santiago	Català	Aula IA	
d00	dl.	dt.	dc.	dj.	dv.	1r sem.	14.00-15.30	Escalera Guerrero, Sergio	Castellà	Aula IG	
f00	dl.	dt.	dc.	dj.	dv.	1r sem.	11.00-12.30	Pujol Vila, Oriol	Català	Aula IB	
Exàmens : 1r parcial [Presencial]											
G1	3 de novembre de 2026.					18.00-21.00	Pujol Vila, Oriol Segui Mesquida, Santiago Escalera Guerrero, Sergio Martínez Martínez, PABLO Hernández Alonso, Manuel Andrés				
Exàmens : Final [Presencial]											
G1	13 de gener de 2027.					15.00-20.00	Pujol Vila, Oriol Segui Mesquida, Santiago Escalera Guerrero, Sergio Martínez Martínez, PABLO Hernández Alonso, Manuel Andrés				
Exàmens : Revaluació [Presencial]											
G1	27 de gener de 2027.					09.00-14.00	Pujol Vila, Oriol Segui Mesquida, Santiago Escalera Guerrero, Sergio				

Llenguatge de programació?

- Python 3

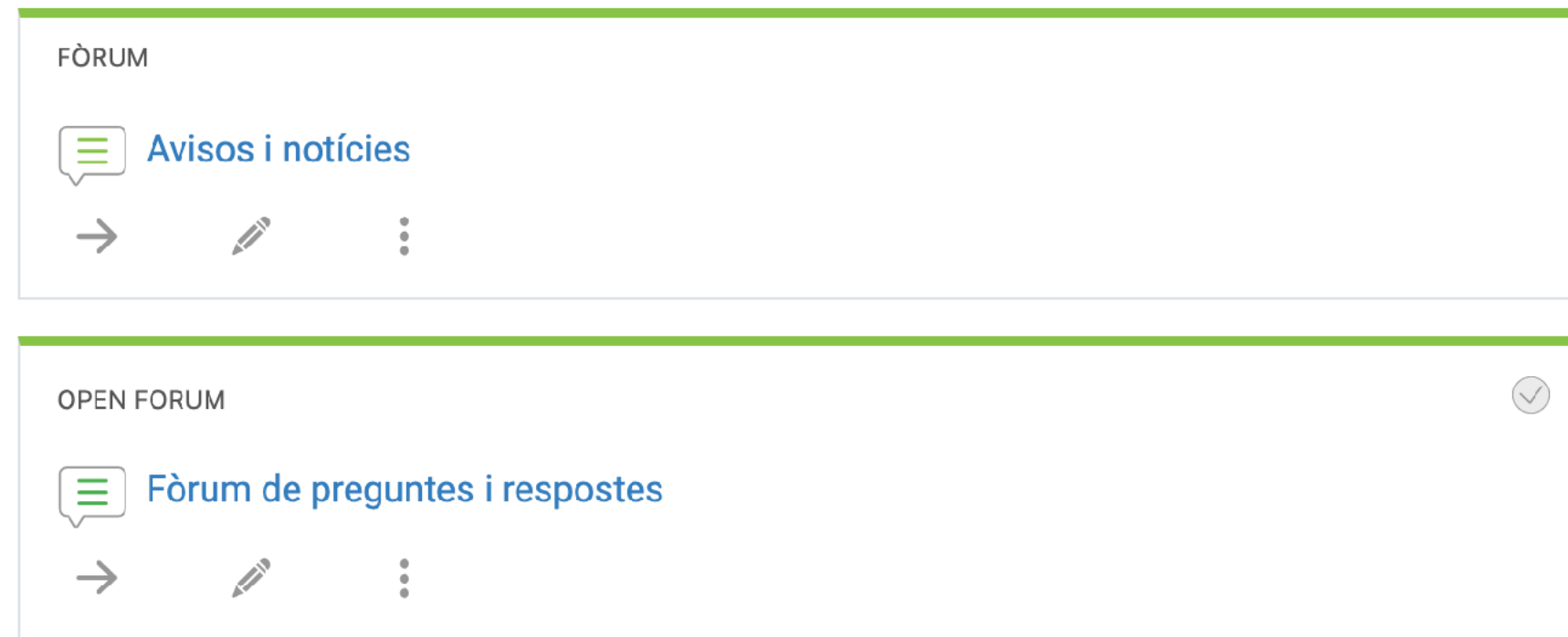
Com s'organitza l'assignatura?

- L'assignatura s'imparteix en classes teòriques, problemes i pràctiques. L'assignatura es coordinarà mitjançant el Campus Virtual. A través d'aquest entorn tindreu: anuncis, apunts, notes, fòrum, calendari, enllaços a la bibliografia, etc.
 - <https://campusvirtual2.ub.edu/>

IMPORTANT: TOTS els dubtes, qüestions, problemes, etc.. de caire no personal s'han de realitzar mitjançant el **Fòrum** del **Campus Virtual**.

No es contestarà cap correu electrònic que no sigui una qüestió personal.

Fòrum



IMPORTANT: TOTS els dubtes, qüestions, problemes, etc.. de caire no personal, s'han de realitzar mitjançant el **Fòrum** del **Campus Virtual**.

No es contestarà cap correu electrònic que no sigui una qüestió personal.

Què puc preguntar?

- Qualsevol cosa referent a la assignatura! Dubtes, problemes amb codis, ...

Puc compartir informació?

- Podeu compartir qualsevol cosa que vulgueu. Entre elles documents, vídeos, solucions a problemes, exàmens d'anys passats solucionats, etc... És a dir, qualsevol cosa que creieu que pot ajudar a la resta d'alumnes.

Segur que puc compartir tot el que vulgui?

- **No està permès** compartir les vostres pràctiques/problemes avaluable, així com qualsevol solució a les pràctiques/problemes avaluable de l'assignatura.

Puc contestar?

- Si!!! Una activa participació al fòrum es tindrà en compte de forma positiva (un 8.8 de l'assignatura és pot convertir amb un 9).

Com s'avaluarà l'assignatura?

- L'assignatura seguirà un esquema d'avaluació continuada, amb dos elements principals: exàmens presencials i lliurament de pràctiques

Proves presencials:

- Setmana exàmens parcial
 - Examen Parcial Teoria (*EPT*)
 - Examen Parcial Pràctiques (*EPP*)
- Examen final
 - Examen Final Teoria (*EFT*)
 - Examen Final Pràctiques (*EFP*)
 - → A l'examen final es permet recuperar la part (EPT i/o EPP) de l'examen parcial **en cas de suspens**. En cas de presentar-se, es renuncia a la nota del parcial.
- Als exàmens (teoria i pràctiques) **NO ES POT portar apunts** ni utilitzar Internet.
- NOTA: Les pràctiques i proves de pràctiques es faran amb "NOTEBOOK"

Lliurament de pràctiques:

- Lliurament de pràctiques: Cada un dels lliuraments de pràctiques serà avaluat pel professor amb una nota que pot anar de 0 a 10.
- Si l'estudiant no lliura les pràctiques dins del període assenyalat, obtindrà un 0.
- La nota final (*NLP*) de la part de pràctiques és la mitjana de tots els lliuraments.

- Nota Teoria (**NT**): Mitjana dels exàmens de teoria (*NPT* i *NFT*)
- Nota Proves Pràctiques (**NPP**) : Mitjana dels exàmens de pràctiques (*NPP* i *NFP*)
- Nota Lliuraments Pràctiques (**NLP**): Són proves presencials. La nota és la mitjana de totes elles.

- Per poder estimar la **Nota final** de l'assignatura, l'estudiant ha de complir la condició següent:

- **Nota final =**

- 35 % Nota Teoria (**NT**) + 35 % Nota Proves Pràctiques (**NPP**) + 30 % Nota Lliuraments Pràctiques (**NLP**)

- Per aprovar l'assignatura s'ha de complir que
 - **Nota final** ≥ 5 ;
 - **NT, NPP i NLP** han ser iguals o superior a 4.
- En cas que **Nota final** ≥ 3.5 l'estudiant té dret a revaluació

Avaluació Única

- L'estudiant que es vulgui acollir a l'avaluació única ho ha de sol·licitar a la Secretaria de la Facultat dins dels terminis establerts dins el curs acadèmic.
- L'avaluació única consisteix en un examen final únic que consta d'una part teòrica (ET) i una prova pràctica (PP) davant de l'ordinador.
- Per aprovar l'assignatura s'ha de complir que
 - **Nota final** ≥ 5 ;
 - **ET i PP** han de ser igual o superior a 4.
- En aquest cas, la nota s'avalua de la manera següent:
 - **Nota_Final** = $0,5 * ET + 0,5 * PP$

Revaluació

- L'alumne té dret a la revaluació sempre que la **Nota Final** obtinguda sigui com a mínim un 3.5.
- La revaluació consta d'un examen **ÚNIC** de teoria (50%) i un examen pràctic (50%).
 - No es conserva cap nota dels exàmens anteriors.
- En el cas de revaluació, les notes de pràctiques no tenen cap pes per a la Nota Final.

Tutories

- Els professors de pràctiques us informaran dels seus horaris de tutories.
- És important enviar un correu al professor abans per concertar l'horari de la consulta.

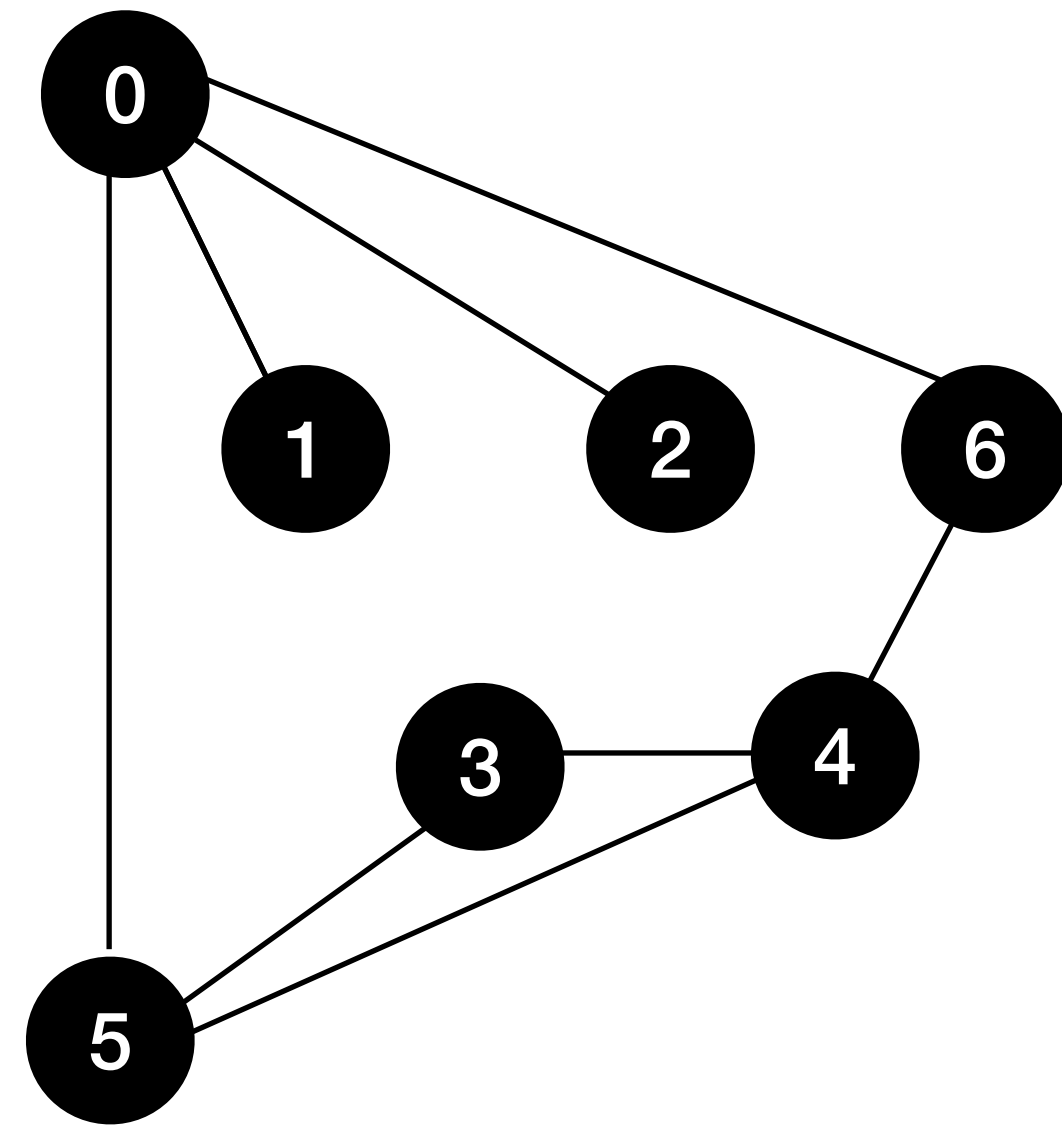
Puc fer servir la IA (ChatGPT, Claude, Copilot)

- Sí, com a eina d'aprenentatge: per entendre conceptes, explorar idees, depurar i accelerar-te un cop entens què fas.
- Avaluem el teu criteri: dissenyar, saber per què i quan un algoritme funciona, validar-lo i triar la millor opció entre alternatives.
- No permès: lliurar practiques/problemes/examens avaluables resolts amb IA que no sapiguem explicar ni entendre.
- Als exàmens: sense Internet ni IA (sempre que el professor no digui el contrari)
- Regla pràctica: Si no pots explicar línia a línia el que has lliurat, el còdi no es teu.

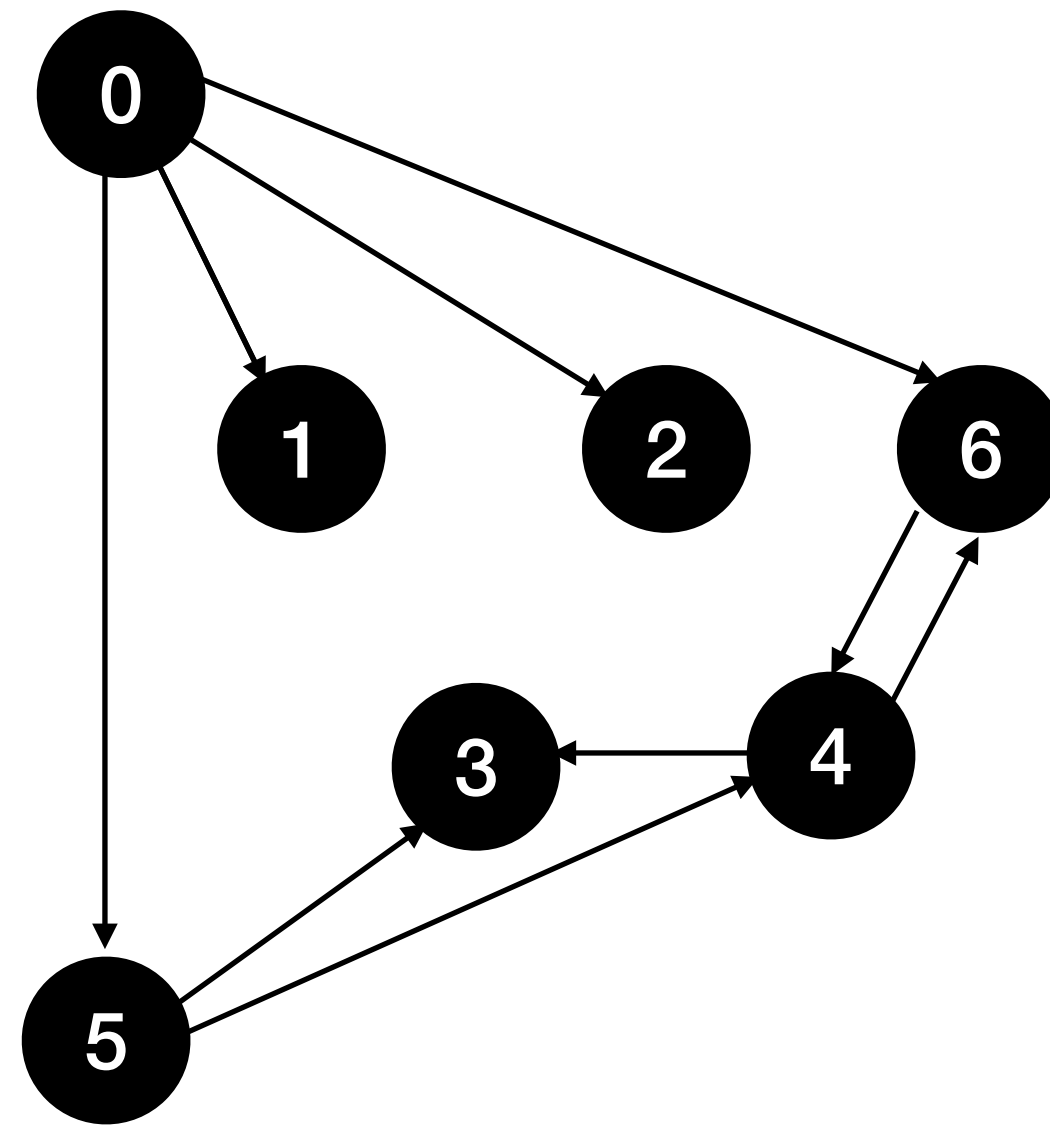
	Teoria	Problemes		Pràctiques		
Setmana	Dilluns	Dimarts	Dijous	Dimarts	Dijous	
14 set	Intro + Grafs I (Representació + DFS)	Introducció		Prac. Introductòria + Prac Graf		
21 set.	Grafs II (BFS, Components Connexes, Dijkstra)					
28 set.	Grafs III (Bellman Ford, Ordre topològic)			Prac Grafs		
5 oct.	Grafs IV (Ford Fulkensen, Min-Cut Max-Flow)			Prac Grafs		
12 oct.	FESTIU			Entrega/Prova Prac Grafs		
19 oct.	Greedy I Minimum Spanning Tree, Kruskal, Union-Find	Greedy I		Prac Greedy		
26 oct.	Greedy II Prim, Graph Coloring, Hoffman Code					
2 nov.	Setmana Exàmens	Setmana Exàmens				
9 nov.	Programació Dinàmica I Fibonacci, Floyd Warshall, Motxilla, Descens riu	Entrega/Prova Greedy				
16 nov.	Programació Dinàmica II LIS, Levenshtein, Problema del Viatjant Comerç	Prac Programació Dinàmica				
23 nov.	Enumeratius I: Backtracking Motxilla, Sudoku, Pintar Mapa	Prac Enumeratius				
30 nov.	Enumeratius II: Branch & Bound Motxilla, Tasques, Puzzle	Prac Enumeratius				
7 des.	FESTIU	FESTIU				
14 des.	Enumeratius III: Branch & Bound TSP	Enumeratius II		Entrega/Prova PD & Enumeratius		
21 des.	Dubtes					

Representació dels grafs

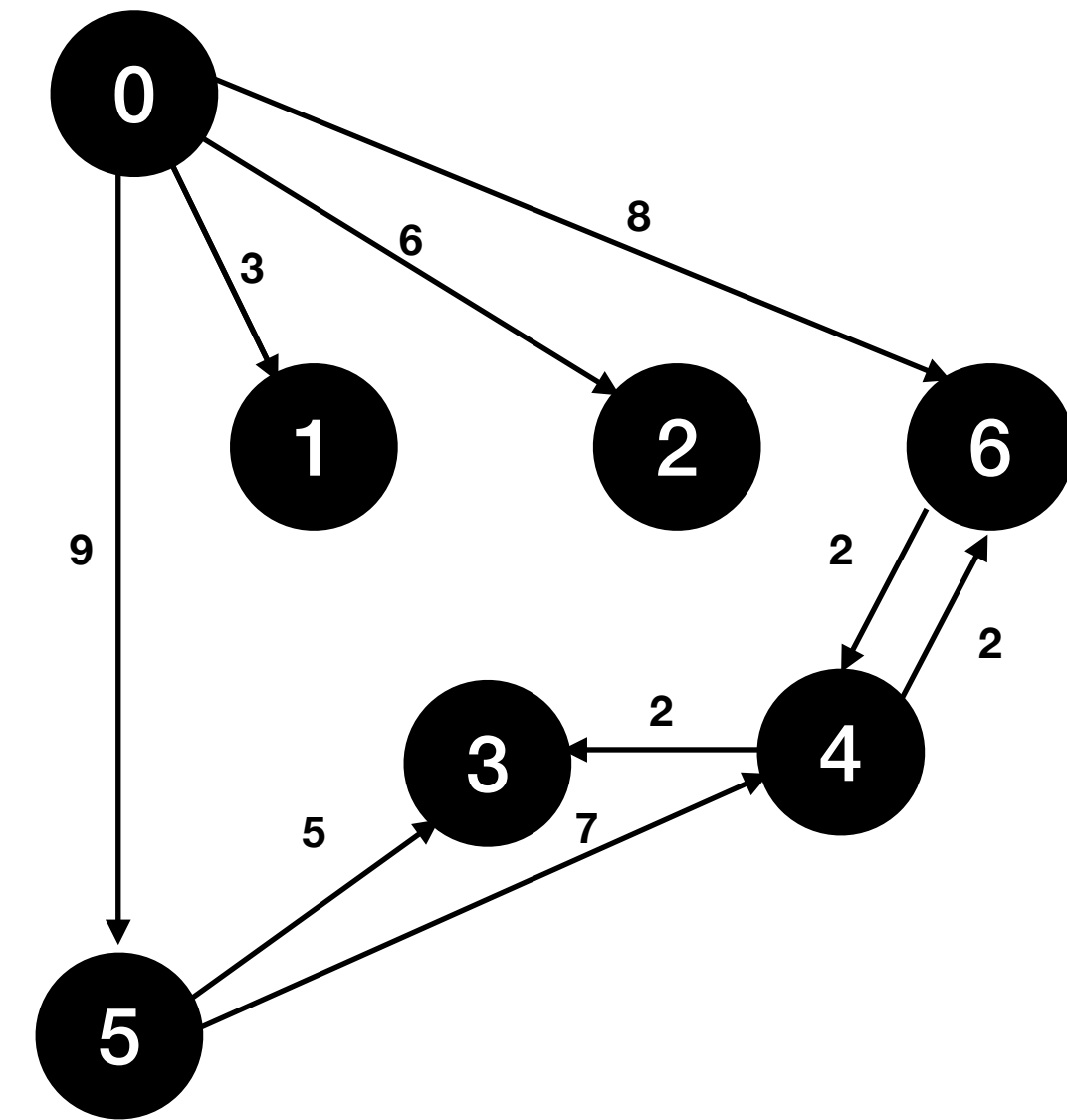
Representació dels grafos



Graf no dirigit



Graf dirigit



Graf dirigit amb pesos

Com representem aquests grafos $G(V, E)$?

Representació dels grafs

- Com representem un graf?
 - Linked list o array
 - Matriu d'adjacència
 - Llista d'adjacència

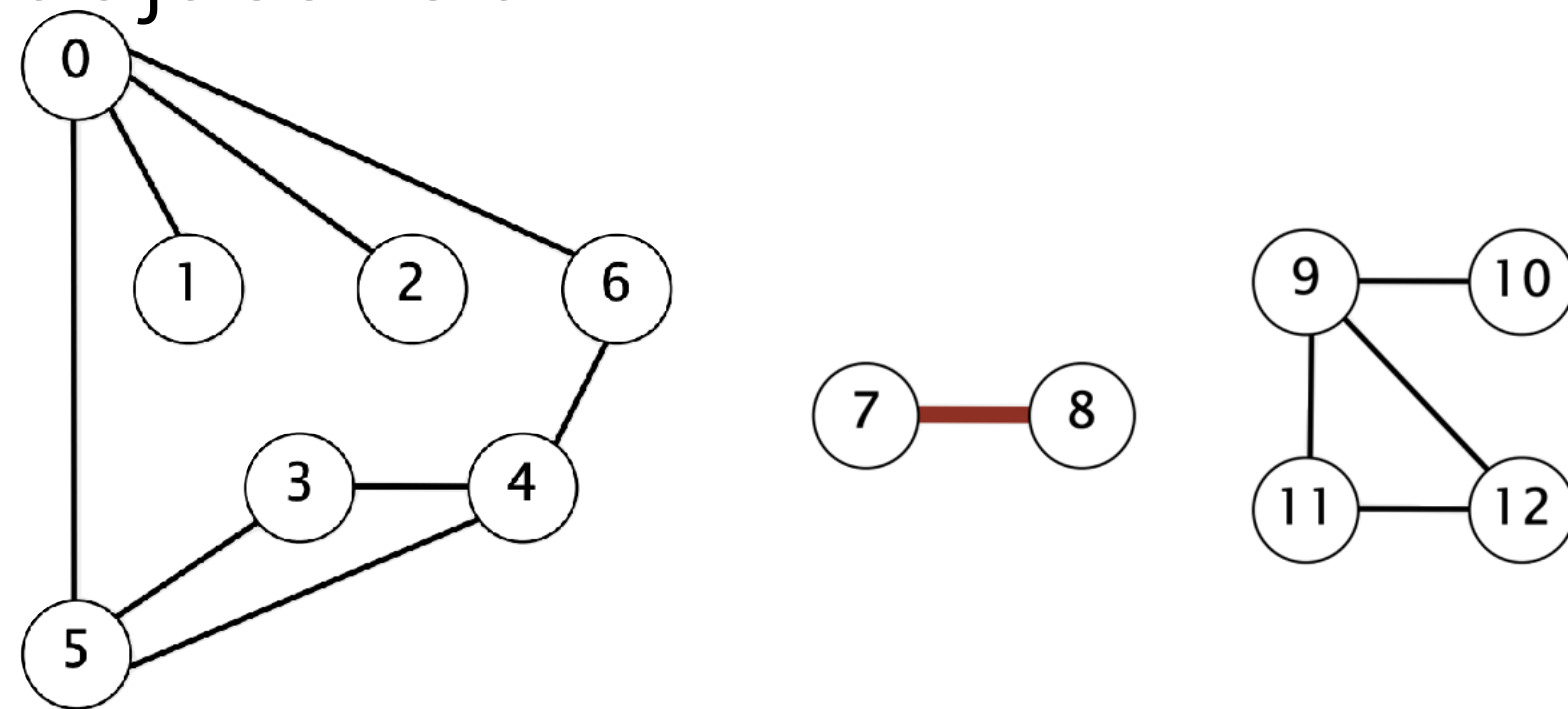
Representació dels grafos

- Com representem un graf?

- **Linked list o array**

- Matriu d'adjacència

- Llista d'adjacència



0	1
0	2
0	5
0	6
3	4
3	5
4	5
4	6
7	8
9	10
9	11
9	12
11	12

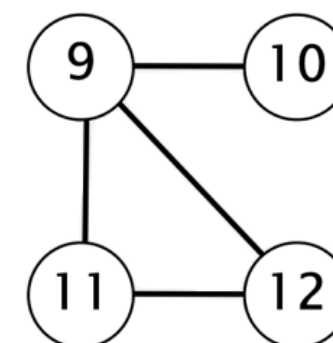
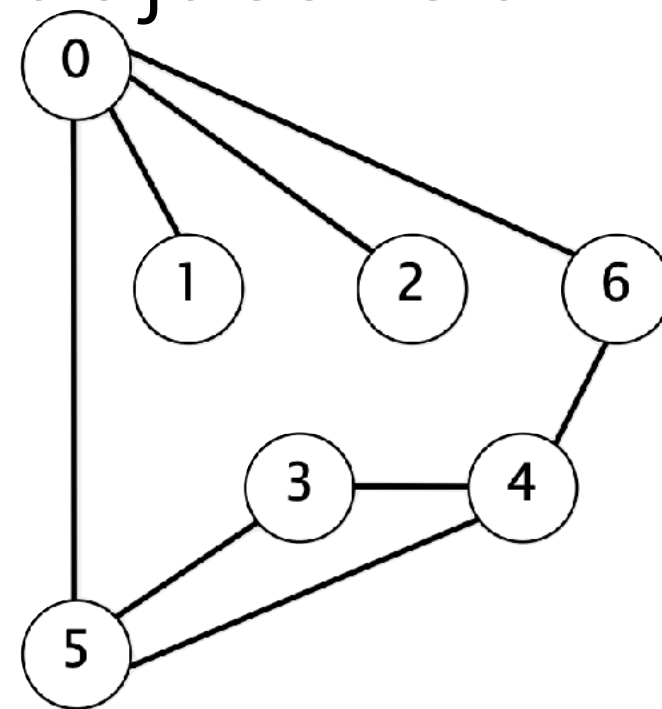
Representació dels grafs

- Com representem un graf?

- Linked list o array

- **Matriu d'adjacència**

- Llista d'adjacència



En cas de graf no dirigit:
La mateixa dada

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0

Representació dels grafos

- Com representem un graf?

- Linked list o array

- **Matriu d'adjacència**

- Llista d'adjacència

Que guardem per a les arestes?

- 1/0 o True/false
- Pes/cost de l'aresta
 - utilitzem valors ficticis a les posicions on no tenim aresta $\{-\infty, 0, \infty\}$
- Objecte de l'aresta

	1	2	3	4	5	6	7
1	∞	1	∞	∞	∞	∞	∞
2	∞	∞	∞	3	∞	15	∞
3	23	3	∞	8	∞	∞	∞
4	∞	∞	3	∞	∞	5	12
5	∞	∞	∞	∞	∞	∞	55
6	∞	1	4	4	∞	∞	
7	∞	∞	∞	∞	∞	6	∞

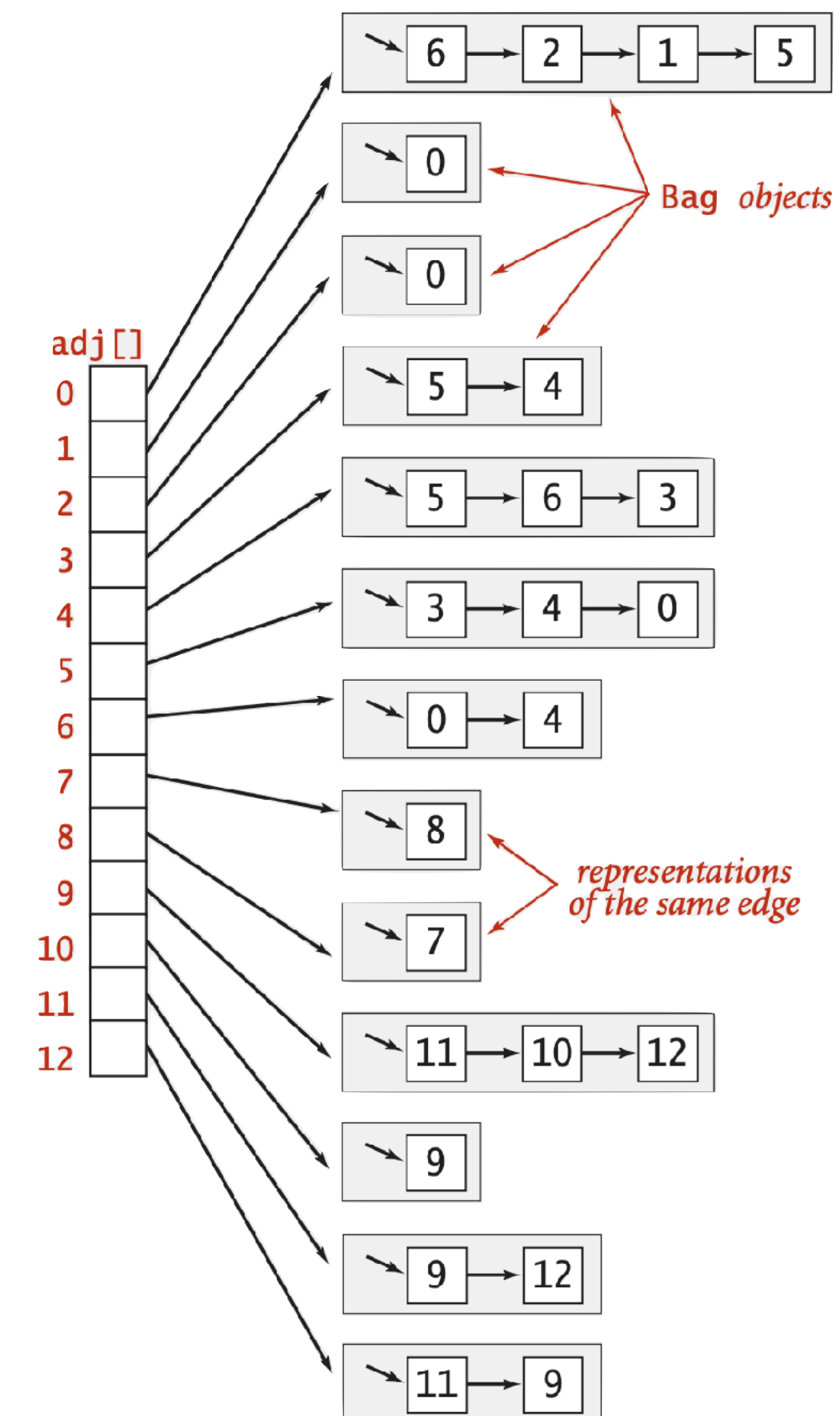
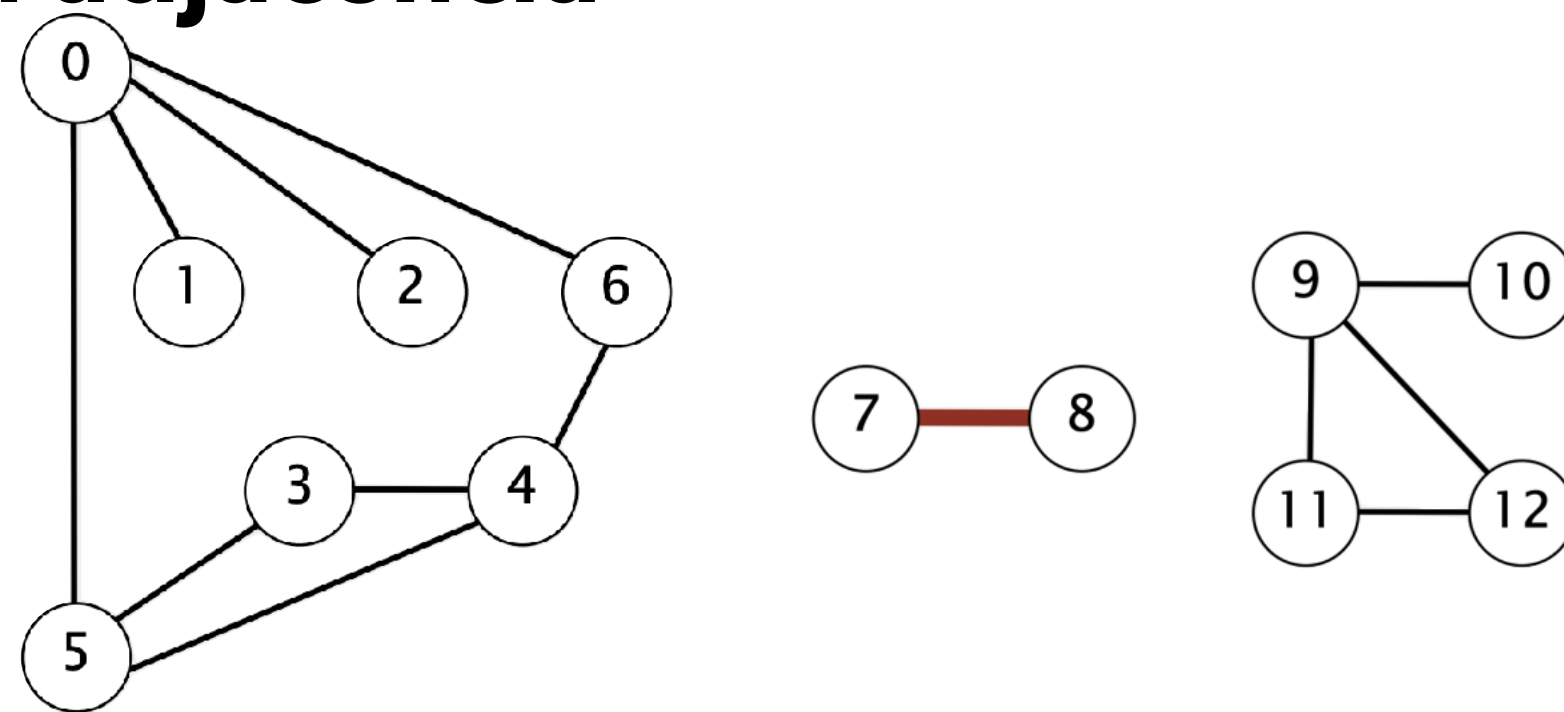
Representació dels grafs

- Com representem un graf?

- Linked list o array

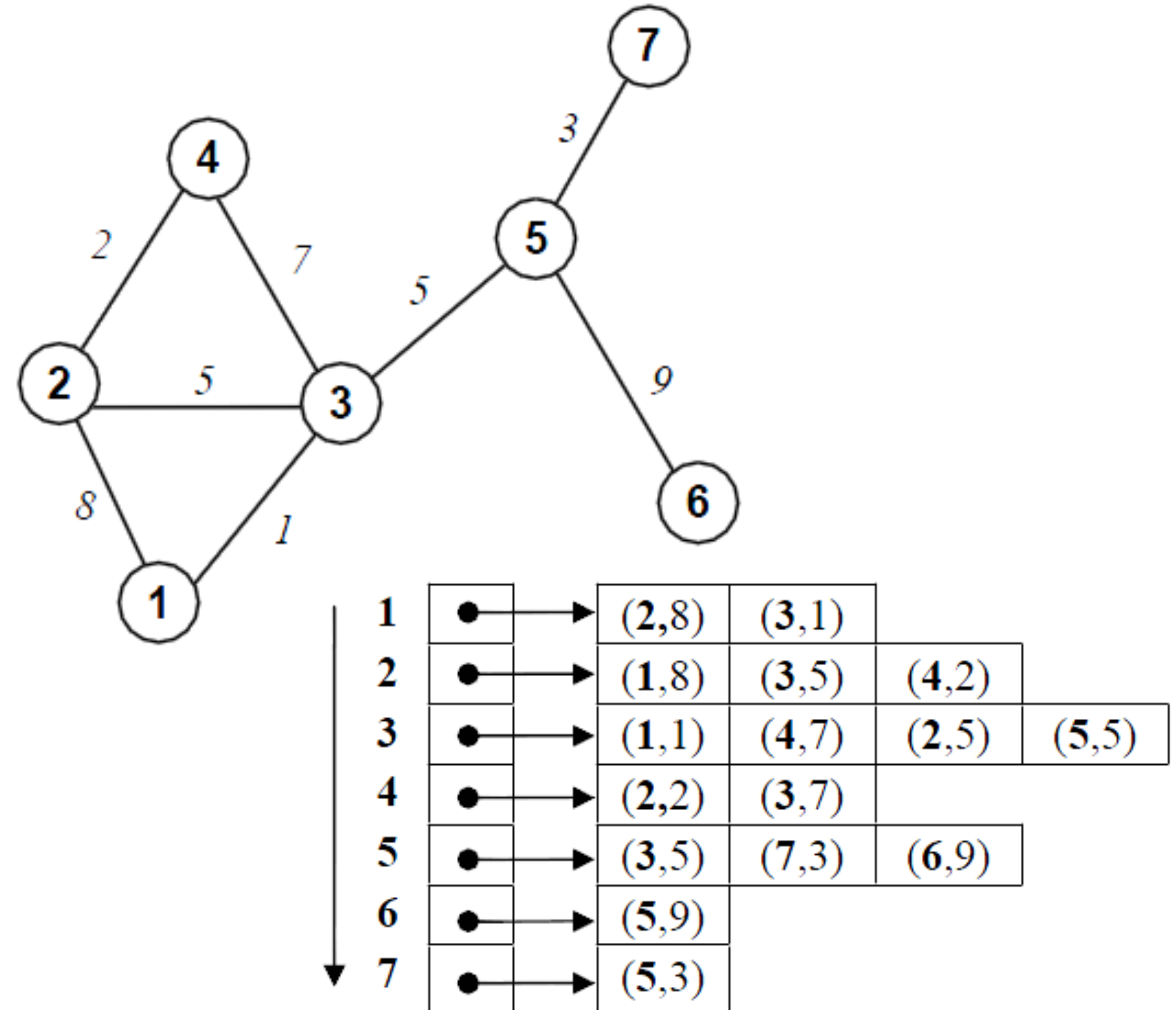
- Matriu d'adjacència

- **Llista d'adjacència**



Representació dels grafs

- Com representem un graf?
- Linked list o array
- Matriu d'adjacència
- **Llista d'adjacència**



Algoritmes sobre grafs

- Matriu ó llista d'adjacència?
- Matriu $\rightarrow |V|^2$ posicions $\rightarrow$ un accés
- Llista $\rightarrow |E|$ posicions $\rightarrow$ mínim un accés memòria versus localització
- Graf **dens** versus graf **sparse**



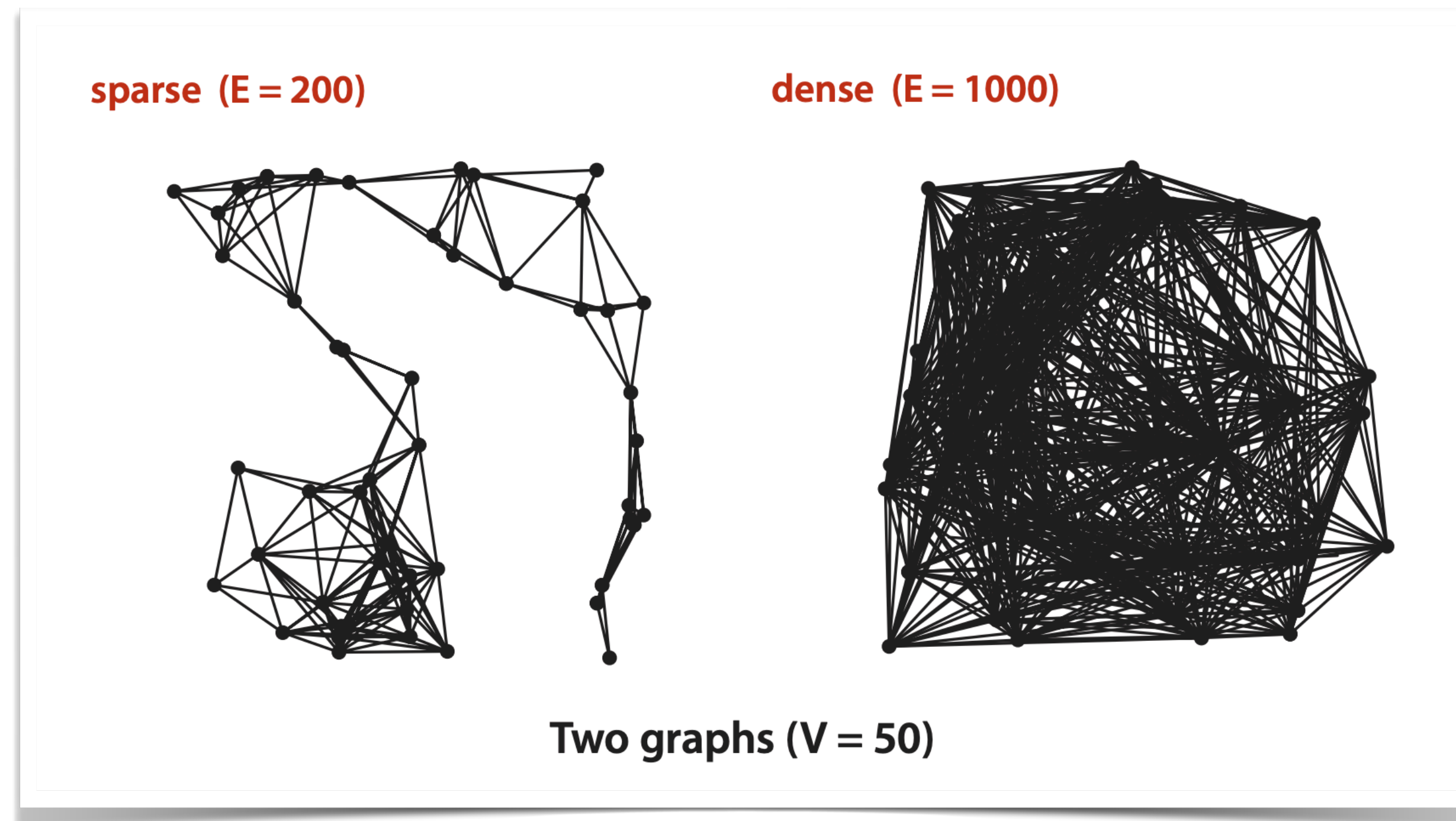
Què faríeu servir per codificar tots els enllaços de les pàgines del www?

Representació dels grafs

- **A la pràctica:** Quin utilitzar?

- Els grafs reals acostumen a ser ***dispersos (sparse)***

Alt nombre de nodes, grau mitjà del nodes petit



Representació dels grafs

- **A la pràctica:** Quin utilitzar?
- Els algoritmes es basen en iterar sobre els nodes adjacents del *vèrtex*
- Els grafs reals acostumen a ser ***sparse***

Alt nombre de nodes, grau mitja del nodes petit

representation	space	add edge	edge between v and w?	iterate over vertices adjacent to v?
list of edges				
adjacency matrix				
adjacency lists				

Representació dels grafes

- **A la pràctica:** Quin utilitzar?
- Els algoritmes es basen en iterar sobre els nodes adjacents del *vèrtex*
- Els grafes reals acostumen a ser ***sparse***

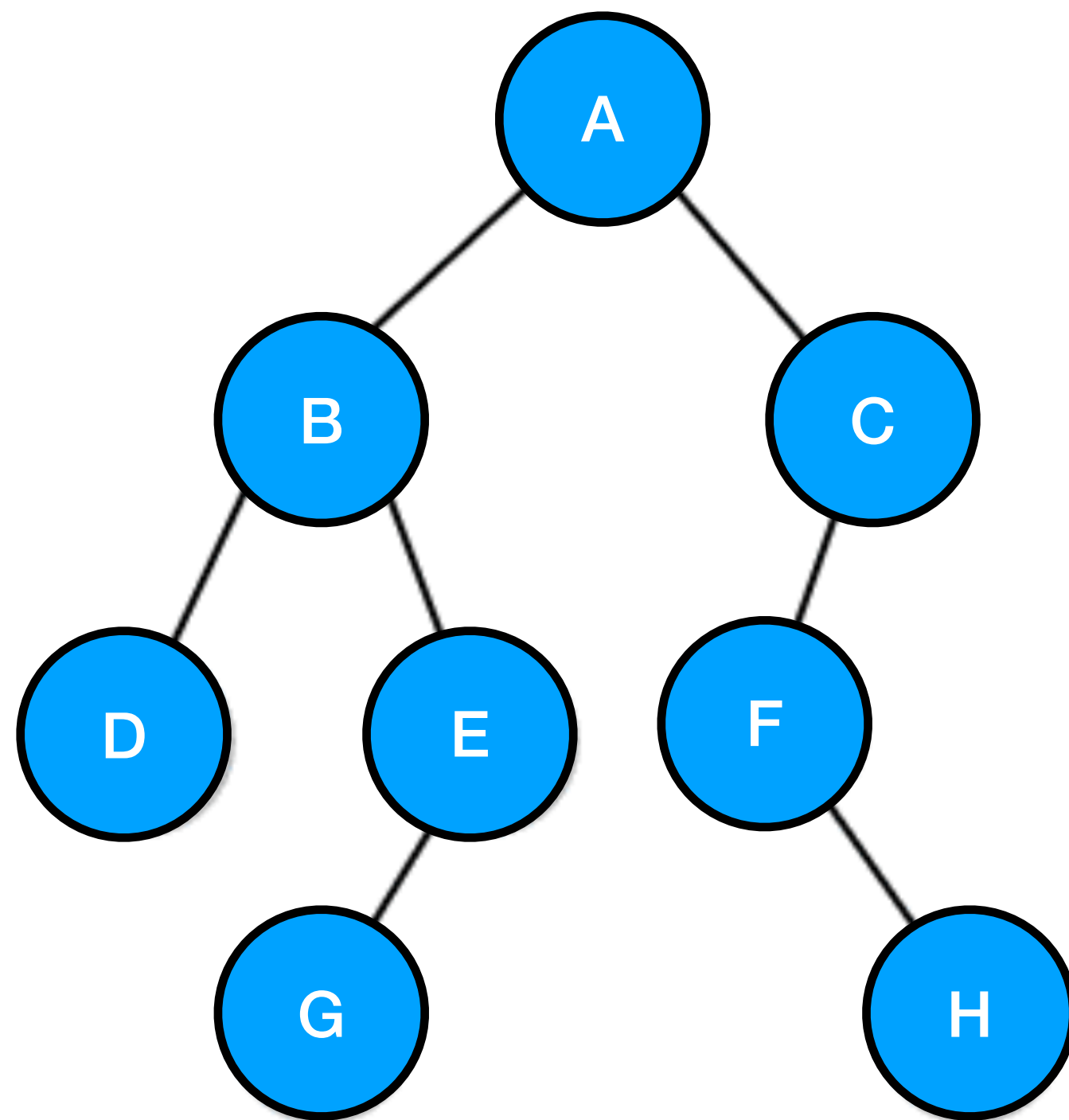
Alt nombre de nodes, grau mitja del nodes petit

representation	space	add edge	edge between v and w?	iterate over vertices adjacent to v?
list of edges	E	1	E	E
adjacency matrix	V^2	1 *	1	V
adjacency lists	$E + V$	1	degree(v)	degree(v)

Recap: Recorregut Arbres

Recap: Recorregut

Text del títol



Donat el següent arbre:

```
# Using a Python dictionary to act as an adjacency list
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['G'],
    'F' : ['H'],
    'G' : [],
    'H' : []
}

visited = set() # Set to keep track of visited nodes.

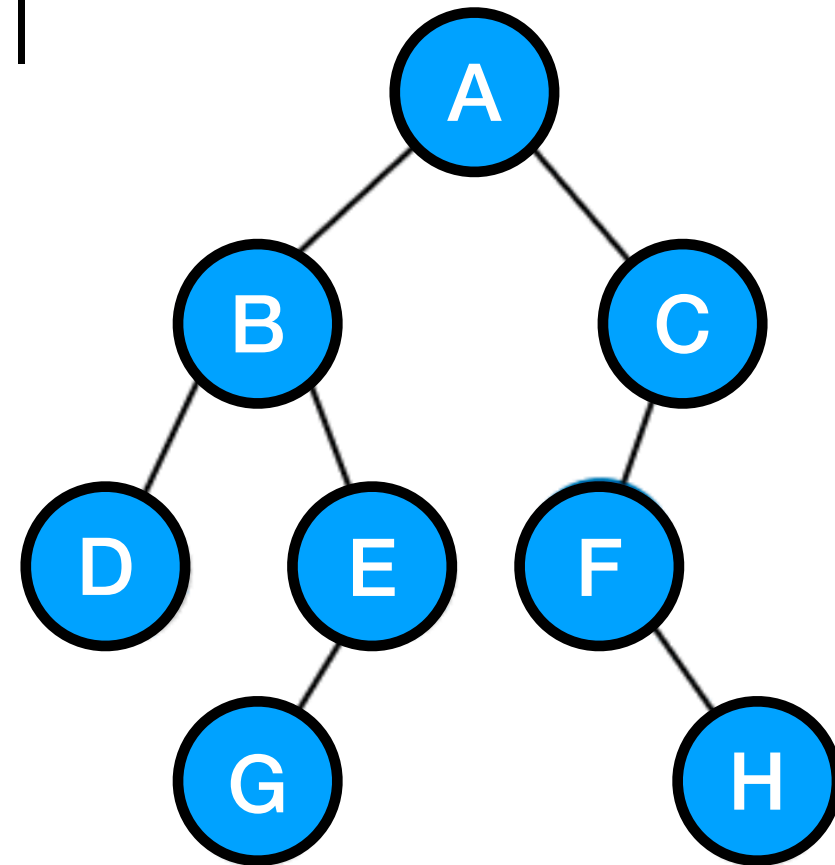
def explore(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            explore(visited, graph, neighbour)

# Driver Code
explore(visited, graph, 'A')
```

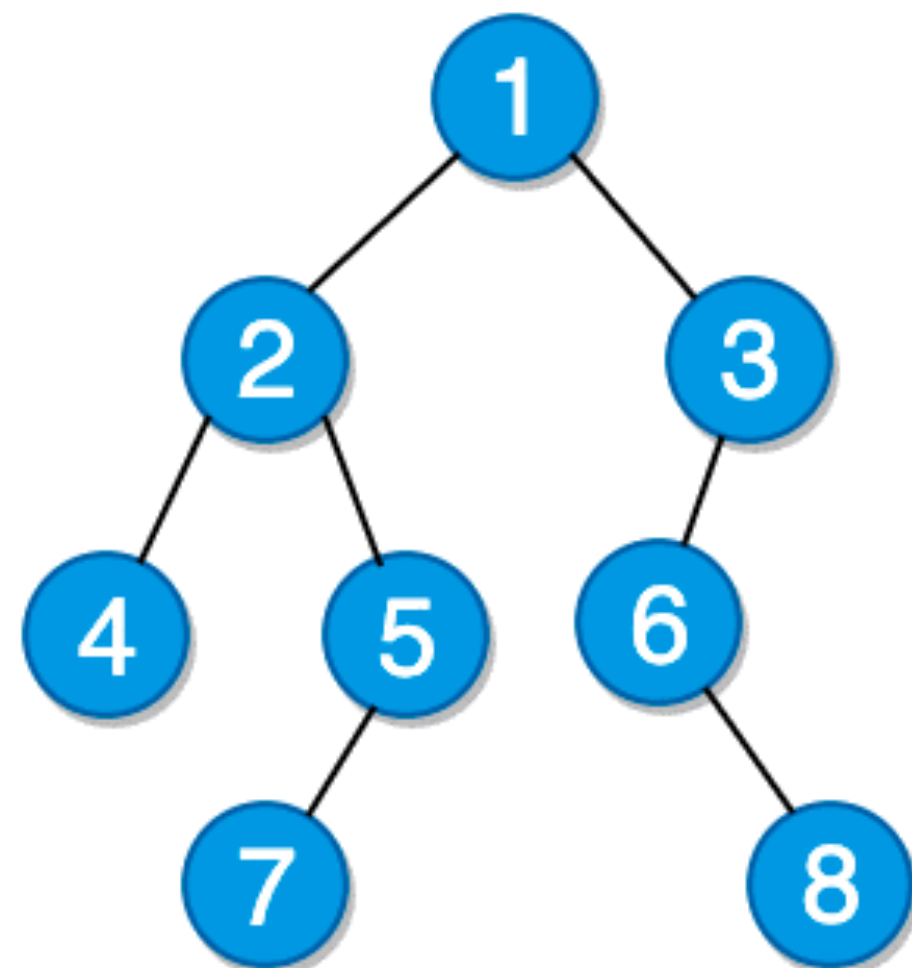
Quina és la sortida del codi anterior? Què fa?

Recap: Recorregut

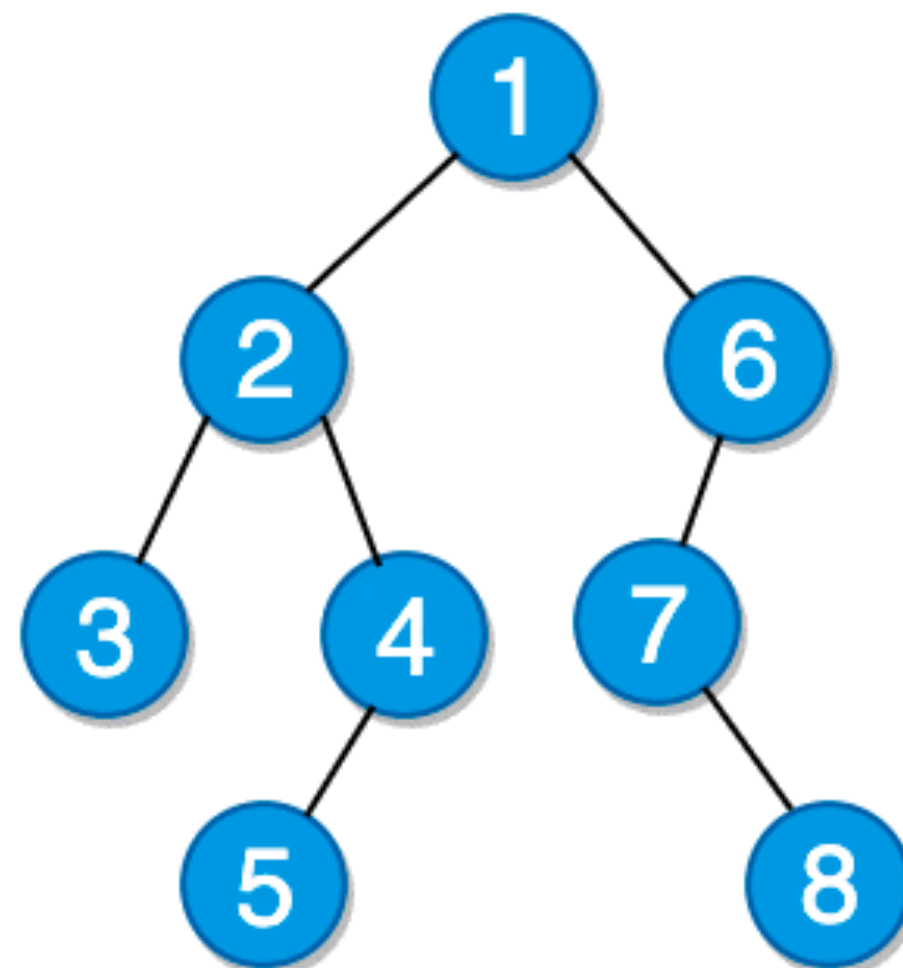
Text del títol



BFS



DFS



Donat el següent arbre:

```
# Using a Python dictionary to act as an adjacency list
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['G'],
    'F' : ['H'],
    'G' : [],
    'H' : []
}
```

```
visited = set() # Set to keep track of visited nodes.
```

```
def explore(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            explore(visited, graph, neighbour)
```

```
# Driver Code
explore(visited, graph, 'A')
```

Quina és la sortida del codi anterior? Què fa?